



2025 金三银四面面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析

1. 往期部分专题

- 目以终为始，重回前端 -- Web Development Trend
- 目以终为始，重回前端 -- Typescript: JavaScript with syntax for types
- 目以终为始，重回前端 -- Awesome JavaScript & Design Patterns
- 目以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade
- 目以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术
- 目以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧
- 目以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库

2. 2025前端早鸟票--今年的金三银四应该如何准备？

| <https://leetcode.com/studyplan/top-interview-150/>

HTML CSS JS

DOM VDOM 浏览器机制

re-render setState

VDOM compiler babel swc
runtime jsx-runtime vue-runtime -> diff
bundler vite

this event loop extend prototype scope ESNext

CSS loader (webpack) postcss less saas tailwind
linter stylelint

HTTP 跨域 安全性
cache CORS ws stream SSE XSS CSRF SQL DDOS

数据结构 & 算法 数组 链表 栈 树
DP

1. 框架 & 工具库 -> 技术方案

1. 多人 巨石 -> 微前端

2. vue react

3. 组件库 hooks库

4. 状态管理 路由

5. esbuild swc vite webpack

6. 定制化

7. CI/CD 配置化 自动化 CLI

8. 稳定性 监控 action
- 痛点难点 具象化

1. 多人协同 -> workflow 系统稳定性

2. 开发效率 -> 标准 & auto

3. 性能 -> 首屏 包体积

方案调研

业务/技术结果

1. 没写过对应的场景题

2. api 不熟练

3. 应用不起来 最优解
- 写得少

Promise Promise.finally

```
Promise.prototype.finally = function (onSettled) {
  return this.then(
    (data) => {
      onSettled(); // 实现了收不到参数了
      return data;
    },
    (reason) => {
      onSettled();
      throw reason;
    }
  );
};
// finally函数 返回结果应该是无效的
}
```

```
const pro = new Promise((resolve, reject) => {
  resolve(1);
});
const pro2 = pro.finally((d) => {
  console.log("finally", d); // 收不到d参数
  // 本身不改变状态，但是抛出一个错误，数据就会变成它的错误
  // throw 123;
  return 123; //不起作用
});
```

```
allSettled
/**
 * 等待所有的Promise有结果后
 * 该方法返回的Promise完成
 * 并且按照顺序将所有结果汇总
 * @param {Iterable} proms
 */
Promise.allSettled=function(proms){
  const ps = [];
  for (const p of proms) {
    ps.push(
      MyPromise.resolve(p).then(
        (value) => ({
          status: FULFILLED,
          value,
        })),
        (reason) => ({
          status: REJECTED,
          reason,
        })
      )
    );
  }
  return MyPromise.all(ps);
}
```

2.1 前端专业知识相关面试考察点

首先我们会针对前端开发相关来介绍需要掌握的一些知识，内容会包括 Javascript、HTML 与 CSS、网络相关、浏览器相关、安全相关、算法和计算机通用知识。

2.1.1 HTML 与 CSS

关于 HTML 的内容会较少单独地问，更多是结合浏览器机制等一起考察：

- DOM 操作是否会带来性能问题
- 事件冒泡/事件委托

关于 CSS，也有以下的一些考察点：

- 介绍盒子模型
- 内联元素与块状元素、display
- 文档流的理解：static / relative / absolute / fixed 等
- 元素堆叠：z-index 与 position 的作用关系
- Flex 布局方式的理解和使用

- Grid 布局方式的理解和使用
- BFC 的优点和缺点
- CSS 动画考察：关键帧、`animate`、`transition` 等

很多时候，面试官也会通过让候选人编码实现某些样式/元素的方式，来考察候选人对 CSS 的掌握程度，其中布局（居中、对齐等）会更容易考察到。

2.1.2 Javascript

前端最基础的技能包括 Javascript、CSS 和 HTML，尤其是新人比较容易遇到这方面的考察。对于 Javascript 会问到多一些，通常包括：

考察范围	具体问题
对单线程 Javascript 的理解	单线程来源 Web Workers 和 Service Workers 的理解
异步事件机制	为什么使用异步事件机制 在实际使用中异步事件可能会导致什么问题 关于 <code>setTimeout</code> 、 <code>setInterval</code> 的时间精确性
对 EventLoop 的理解	介绍浏览器的 EventLoop 宏任务（MacroTask）和微任务（MicroTask）的区别 <code>setTimeout</code> 、 <code>Promise</code> 、 <code>async/await</code> 在不同浏览器的执行顺序
Javascript 的原型和继承	如何理解 Javascript 中的“一切皆对象” 如何创建一个对象 <code>proto</code> 与 <code>prototype</code> 的区别
作用域与闭包	请描述以下代码的执行输出内容（考察作用域） 什么场景需要使用闭包 闭包的缺陷
this与执行上下文	简单描述 <code>this</code> 在不同场景下的指向 <code>apply/call/bind</code> 的使用 箭头函数与普通函数的区别
ES6+	对 <code>Promise</code> 的理解 使用 <code>async</code> 、 <code>await</code> 的好处 浏览器兼容性与 Babel <code>Set</code> 和 <code>Map</code> 数据结构

对 Javascript 的考察，也可以通过写代码的方式来进行，例如：

- 手写代码实现 `call` / `apply` / `bind`
- 手写代码实现 `Promise`、`async` / `await`
-Javascript 中 `0.1+0.2` 为什么等于 `0.30000000000000004`，如何通过代码解决这个问题

2.1.3 网络相关

网络相关的知识在日常开发中也是挺常用的，相关的问题可以从“一个完整的 HTTP 请求过程”来讲述，包括：

- 域名解析（此处涉及 DNS 的寻址过程）
- TCP 的 3 次握手
- 建立 TCP 连接后发起 HTTP 请求
- 服务器响应 HTTP 请求

以上的内容都需要尽数掌握，除此以外，关于 HTTP 的还有以下常见内容：

- HTTP 消息的结构，包括 Request 请求、Response 响应
- HTTP 请求方法，使用 PUT、DELETE 等方法时为什么有时候在浏览器会看到两次请求（涉及 CROS 中的简单请求和复杂请求）
- 常见的 HTTP 状态码

- 浏览器是如何控制缓存的：相应的头信息、状态码
- 如何对请求进行抓包和改造
- Websocket 与 HTTP 请求的区别
- HTTPS、HTTP2 与 HTTP 的对比
- 网络请求的优化方法

2.1.4 浏览器相关

关于浏览器，有很多的机制需要掌握。通常来说，面试官会从一个叫“在浏览器里面输入 URL，按下回车键，会发生什么？”中进行考察，首先会经过上面提到的 HTTP 请求过程，然后还会涉及以下内容：

考察内容	相关问题
浏览器的同源策略	“同源”指什么 那些行为受到同源策略的限制 常见的跨域方案有哪些
浏览器的缓存相关	Web 缓存通常包括哪些 浏览器什么情况下会使用本地缓存 强缓存和协商缓存的区别 强制ctrl+F5刷新会发生什么 session、cookie 以及 storage 的区别
浏览器加载顺序	为什么我们通常将 Javascript 放在<body>的最后面 为什么我们将 CSS 放在<head>里
浏览器的渲染原理	HTML/CSS/JS 的解析过程 渲染树是怎样生成的 重排和重绘是怎样的过程 日常开发中要注意哪些渲染性能问题
虚拟 DOM 机制	为什么要使用虚拟 DOM 为什么要使用 Javascript 对象来描述 DOM 结构 简单描述下虚拟 DOM 的实现原理
浏览器事件	DOM 事件流包括几个阶段（点击后会发生什么） 事件委托是什么

2.1.5 安全相关

安全在实际开发中是最重要的，作为前端开发，同样需要掌握：

- 前端安全中，需要注意的有哪些问题
- XSS/CSRF 是怎样的攻击过程，要怎么防范
- 除了 XSS 和 CSRF，你还了解哪些网络安全相关的问题呢
- SQL 注入、命令行注入是怎样进行的
- DDoS 攻击是什么
- 流量劫持包括哪些内容

2.1.6 算法与数据结构

很多大公司会考察算法基础，大家其实也可以多上 leetcode 去刷题，这些题目刷多了就有感觉了。前端比较爱考的包括：

- 各种排序算法、稳定排序与原地排序、JS 中的 sort 使用的是什么排序
- 查找算法（顺序、二分查找）
- 递归、分治的理解和应用
- 动态规划

除此之外，常见的数据结构也需要掌握：

- 链表与数组
- 栈与队列
- 二叉树、平衡树、红黑树等

很多人会觉得，对前端开发来说算法好像并不那么重要，的确日常开发中也几乎用不到。但不管是前端开发也好，还是后台开发、大数据开发等，软件设计很多都是相通的。一些比较著名的前端项目中，也的确会用到一些算法，同样树状数据结构其实也在前端中比较常见。

2.1.7 计算机通用知识

同样的，虽然在日常工作中我们接触到的内容比较局限于前端开发，但以下内容作为开发必备基础，也是需要掌握的：

- 理解计算机资源，认识进程与线程（单线程、单进程、多线程、多进程）
- 了解阻塞与非阻塞、同步与异步任务等
- 进程间通信（IPC）常包括哪些方式，进程间同步机制又包括哪些方式
- Socket 与网络进程通信是怎样的关系、Socket 连接过程是怎样的
- 简单了解数据库（事务、索引）
- 常见的设计模式有哪些、列举实际使用过的一些设计模式
- 如何理解面向对象编程、对函数式编程的看法

基础知识相关的内容真的不少，但是这块其实只要准备足够充分就可以掌握。参加过高考的我们，理解和记忆这么些内容，其实没有想象中那么难的。

2.2 前端项目经验相关面试考察点

项目经验通常和个人经历关系比较大，前端业务相关的的一些项目经验通常包括管理端、H5 直出、Node.js、可视化，另外还包括参与工具开发的经验，方案选型、架构设计等。

项目相关的内容，比如性能优化、前端框架之类的

2.2.1 前端框架与工具库

首先我们来看看前端框架，不管你开发管理端、PC Web、H5，还是现在比较流行的小程序，总会面临要使用某一个框架来开发。因此，以下的问题可能与你有关：

- 谈谈你对前端常见的框架（Angular/React/Vue）的理解
- 该项目使用 Angular/React/Vue 的原因是
- 如果现在你重新决策，你会使用什么框架
- 你了解过这些框架都做了哪些事情，介绍一下是怎么实现的
- Vue 中的双向绑定是怎么实现的？
- 讲讲 React 的虚拟 DOM
- 如何进行状态管理，Vuex/Redux/Mobx 等工具是怎么做的
- 单页应用是什么？路由是如何实现的
- 如何进行 SEO 优化

如果你使用到了小程序，还可能会问到：

- 小程序和 H5 有什么不一样，为什么选小程序而不是 H5
- 有考虑在小程序里嵌 H5 实现吗，为什么
- 为什么小程序的性能要好一些
- 小程序开发有用到哪些框架吗、为什么

而工具库相关的就太多了，一般会这么问：

- 有实际使用过哪些第三方库

- 这些工具库有什么特性和优缺点

项目相关的许多问题，其实是我们工作中经常会遇到并需要进行思考的问题。如果平时有养成思考和总结的习惯，那么这些问题很容易就能回答出来。如果平时工作中比较少进行这样的思考，也可以在面试准备的时候多关注下。

2.2.2 Node.js 与服务端

Node.js 相关的可能包括：

- 为什么要用 Node.js（而不是 PHP/JAVA/GO/C++等）
- Node.js 有哪些特点，单线程的优势和缺点是什么
- Node.js 有哪些定时功能
- `Process.nextTick` 和 `setImmediate` 的区别
- Node.js 中的异步和同步怎么理解，异步流程如何控制
- 简单介绍一下 Node.js 中的核心内置类库(事件，流，文件，网络等)
- express 是如何从一个中间件执行到下一个中间件的
- express、koa、egg 之间的区别
- Rest API 有使用过吗，介绍一下

以上这些都属于很基础的问题。很多时候，我们会使用 Node.js 去做一些脚本工程或是服务端接入层等工作。如果项目中有使用 Node.js，面试官更多会结合项目相关的进行提问。

2.2.3 性能优化

性能优化的其实跟项目比较相关，常见的包括：

- 有做过性能优化相关的项目吗，具体的优化过程是怎样的/优化效果是怎样的
- 常见的性能优化包括哪些内容
- 如何理解项目的性能瓶颈/什么时候我们需要对一个项目进行优化
- 图片加载性能有哪些可以优化的地方
- 要怎么做好代码分割/降低代码包大小可以有哪些方式
- 首屏页面加载很慢，要怎么优化
- Tree Shaking 是怎样一个过程
- 页面有没有做什么柔性降级的处理

很多时候，性能优化也是与项目本身紧紧相关，一般来说会包括首屏耗时优化、页面内容渲染耗时优化、内存优化等，可能涉及代码包大小、下载耗时、首屏直出、存储资源（内存/indexDB）等内容。

2.2.4 前端工程化

如今前端工程化的趋势越来越重，通常从脚手架开始：

- 为什么我们开发的时候要使用脚手架
- 如何理解模块化
- 为什么要使用 Webpack，它和 Gulp 的区别是
- 讲一下 Webpack 中常用的一些配置、Loader、插件
- Babel 的作用是什么，如何选择合适的 Babel 版本
- Webpack 是怎么将多个文件打包成一个，依赖问题如何解决
- 有写过 Webpack 插件吗，Webpack 编译的过程具体是怎样的
- CSS 文件打包过程中，如何避免 CSS 全局污染
- 本地开发和代码打包的流程分别是怎样的

除了脚手架相关，如今自动化、流程化的使用也越来越多了：

- 怎么理解持续集成和持续部署
- 你们的项目有使用 CI/CD 吗，为什么
- 你们的代码有写单元测试/自动化测试吗，为什么
- 前端代码支持自动化发布吗，如何做到的

工程化和自动化是如今前端的一个趋势，由于团队协作越来越多，如何提升团队协作的效率也是一个可具备的技能。

2.2.5 开发效率提升

效能提升的意识在工作中很重要，大家都不喜欢低效的加班。通常可能问到的问题包括：

- 做了很多的管理端/H5，有考虑过怎么提升开发效率吗
- 你的项目里，有没有哪些工作是可以的工具完成的
- 项目中有进行组件和公共库的封装吗
- 如何管理这些公共组件/工具的兼容问题
- 日常工作中，如何提升自己的工作效率

2.2.6 监控、灰度与发布

发布和监控这部分，可能较大的业务才会有，涉及的问题可以有：

- 日常开发过程中，怎么保证页面质量
- 版本发布有进行灰度吗？灰度的过程是怎样的
- 版本发布过程中，如何及时地发现问题
- 发生异常，要怎么快速地定位到具体位置
- 如何观察线上代码的运行质量

对于大型项目来说，灰度发布几乎是开发必备，而监控和问题定位也需要各式各样的工具来辅助优化。

2.2.7 多人协作

一些较大的项目，通常由多个开发合作完成。而多人协作的经验也很有帮助：

- 多人开发过程中，代码冲突如何解决
- 项目中有使用 Git 吗？介绍一下 Git flow 流程
- 如果项目频繁交接，如何提升开发效率
- 有遇到代码习惯差异的问题吗，如何解决
- 有哪些常用的代码校验的工具
- 怎么强制进行 Code Review

看到这么多内容不要慌，一般来说面试官只会根据你的工作经历来询问对应的问题，所以如果你并没有完全掌握某一块的内容，请不要写在简历上，你永远也不知道面试官会延伸到哪

3. 场景题千奇百怪，教你以不变应万变？

场景题讲究的是：

1. 很多 call、bind、apply、instanceof 等是上10年前的面试题了，面试官还会问你他最开始学前端时的初级题目吗？
2. 融会贯通，举一反三，难的不是代码，是思路

3.1 Promise (Easy、Medium)

3.1.1 Promise.finally

```

1  /**
2   * 无论成功还是失败都会执行回调
3   * @param {Function} onSettled
4   */
5  Promise.prototype.finally = function (onSettled) {
6      return this.then(
7          (data) => {
8              onSettled(); // 实现了收不到参数了
9              return data;
10         },
11         (reason) => {
12             onSettled();
13             throw reason;
14         }
15     );
16     // finally函数 返回结果应该是无效的
17 }
18
19 /*****test finally*****/
20 // 无论什么结果，都会运行
21 const pro = new Promise((resolve, reject) => {
22     resolve(1);
23 });
24 const pro2 = pro.finally((d) => {
25     console.log("finally", d); // 收不到d参数
26     // 本身不改变状态，但是抛出一个错误，数据就会变成它的错误
27     // throw 123;
28     return 123; //不起作用
29 });
30 setTimeout(() => {
31     console.log(pro2);
32 });
33

```

3.1.2 Promise.allSettled

```

1  /**
2   * 等待所有的Promise有结果后
3   * 该方法返回的Promise完成
4   * 并且按照顺序将所有结果汇总
5   * @param {Iterable} proms
6   */
7  Promise.allSettled=function(proms) {
8      const ps = [];
9      for (const p of proms) {
10         ps.push(
11             MyPromise.resolve(p).then(
12                 (value) => ({
13                     status: FULFILLED,
14                     value,
15                 }),
16                 (reason) => ({
17                     status: REJECTED,
18                     reason,
19                 })
20             )
21         );
22     }
23     return MyPromise.all(ps);
24

```

3.1.3 Promise.all

```

1  /**
2   * 等待所有的Promise有结果后
3   * 该方法返回的Promise完成
4   * 并且按照顺序将所有结果汇总
5   * @param {Iterable} proms
6   */
7  Promise.allSettled=function(proms) {
8    const ps = [];
9    for (const p of proms) {
10     ps.push(
11       MyPromise.resolve(p).then(
12         (value) => ({
13           status: FULFILLED,
14           value,
15         }),
16         (reason) => ({
17           status: REJECTED,
18           reason,
19         })
20       )
21     );
22   }
23   return MyPromise.all(ps);
24 }
```

3.1.4 实现网络请求超时判断，超过三秒视为超时

```

1  function loadImg(src){
2    new Promise((resolve,reject)=>{
3      let img = new Image()
4      img.onload=function(){
5        console.log("图片加载成功")
6        resolve(img)
7      }
8      img.error=function(){
9        reject(new Error(`Can not load ${src}`))
10      }
11      img.src=src
12    })
13  }
14 }
15 function timeout(){
16   return new Promise((reject)=>{
17     setTimeout(()=>{
18       reject("超时")
19     },3000)
20   })
21 }
22 //判断图片加载是否超时
23 Promise.race([loadImg,timeout])
24 .then((data)=>{
25   console.log(data)
26 }).catch(err=>{
```



```
27     console.log(err)
28   })
```

3.1.5 设计一个简单的任务队列, 要求分别在 1,3,4 秒后打印出 "1", "2", "3";

题目

```
1  new Quene()
2    .task(1000, () => {
3      console.log(1)
4    })
5    .task(2000, () => {
6      console.log(2)
7    })
8    .task(1000, () => {
9      console.log(3)
10   })
11   .start()
12
13  function Quene() { ... } //补全代码
```

实现

```
1  function Queue() {
2    this.quene = [];
3  }
4
5
6  Queue.prototype.task = function (time, callback) {
7    this.quene.push({ time, callback });
8    return this;
9  };
10
11 Queue.prototype.start = function () {
12   const quene = this.quene;
13   let result = Promise.resolve();
14
15   quene.forEach((item) => {
16     result = result.then(() => {
17       return new Promise((resolve, reject) => {
18         setTimeout(() => {
19           resolve(item.callback());
20         }, item.time);
21       });
22     });
23   });
24
25   return result;
26 };
27
28 //test
29 new Queue()
30   .task(1000, () => {
31     console.log(1)
32   })
33   .task(2000, () => {
34     console.log(2)
35   })
```

```
36     .task(1000, () => {
37         console.log(3)
38     })
39     .start()
40
```

3.1.6 交通灯

```
1  function red() {
2      console.log('red')
3  }
4
5  function green() {
6      console.log('green')
7  }
8
9  function yellow() {
10     console.log('yellow')
11 }
12 // 要求：红灯3秒亮一次，黄灯2秒亮一次，绿灯1秒亮一次。
13 const light = function(timer, cb) {
14     return new Promise(resolve => {
15         setTimeout(() => {
16             cb()
17             resolve()
18         }, timer);
19     })
20 }
21
22 const step = function() {
23     Promise.resolve().then(() => {
24         return light(3000, red)
25     }).then(() => {
26         return light(2000, green)
27     }).then(() => {
28         return light(1000, yellow)
29     }).then(() => {
30         return step()
31     })
32 }
33
34 step()
```

3.1.7 实现有并行限制的 Promise 调度器

```
1  class Scheduler {
2      constructor(max) {
3          // 最大可并发任务数
4          this.max = max;
5          // 当前并发任务数
6          this.count = 0;
7          // 阻塞的任务队列
8          this.queue = [];
9      }
10
11     async add(fn) {
12         if (this.count >= this.max) {
```

```

13      // 若当前正在执行的任务，达到最大容量max
14      // 阻塞在此处，等待前面的任务执行完毕后将resolve弹出并执行
15      await new Promise(resolve => this.queue.push(resolve));
16  }
17  // 当前并发任务数++
18  this.count++;
19  // 使用await执行此函数
20  const res = await fn();
21  // 执行完毕，当前并发任务数--
22  this.count--;
23  // 若队列中有值，将其resolve弹出，并执行
24  // 以便阻塞的任务，可以正常执行
25  this.queue.length && this.queue.shift()();
26  // 返回函数执行的结果
27  return res;
28  }
29  }
30
31  // 延迟函数
32  const sleep = time => new Promise(resolve => setTimeout(resolve, time));
33
34  // 同时进行的任务最多2个
35  const scheduler = new Scheduler(2);
36
37  // 添加异步任务
38  // time: 任务执行的时间
39  // val: 参数
40  const addTask = (time, val) => {
41    scheduler.add(() => {
42      return sleep(time).then(() => console.log(val));
43    });
44  };
45
46  addTask(1000, '1');
47  addTask(500, '2');
48  addTask(300, '3');
49  addTask(400, '4');
50  // 2
51  // 3
52  // 1
53  // 4

```

3.2 JS基础相关（Easy、Medium）

3.2.1 请实现一个模块 math，支持链式调用math.add(2,4).minus(3).times(2);

```

1  class math {
2    constructor(initValue = 0) {
3      this.value = initValue;
4    }
5    add(...args) {
6      this.value = args.reduce((pre, cur) => pre + cur, this.value);
7      return this;
8    }
9    minus(...args) {
10     this.value = this.value - args.reduce((pre, cur) => pre + cur);
11     return this;
12   }
13   times(timer) {

```

```

14     this.value = timer * this.value;
15     return this;
16 }
17 getVal() {
18     return this.value;
19 }
20 }

```

3.2.2 手写用 ES6proxy 如何实现 arr[-1] 的访问

```

1  let arr= [1,2,3,4]
2  let proxy = new Proxy(arr,{
3      get(target,key){
4          if(key<0){
5              return target[target.length+parseInt(key)]
6          }
7          return target[key]
8      }
9  })
10 console.log(proxy[-2]);
11 console.log(proxy[2]);

```

3.2.3 实现 jsonp

```

1  const jsonp = (url, params, cbName) => {
2      return new Promise((resolve, reject) => {
3          const script = document.createElement("script");
4          window[cbName] = (data) => {
5              resolve(data);
6              document.body.removeChild(script);
7          }
8          params = {...params, callback: cbName};
9          const arr = Object.keys(params).map(key => `${key}=${params[key]}`);
10         script.src = `${url}?${arr.join('&')}`;
11         document.body.appendChild(script);
12     })
13 }

```

3.2.4 设计LRU缓存结构

```

1  // Vue3的keepalive组件就用了这个LRU管理组件的缓存
2  const LRUCache = function (capacity) {
3      this.map = new Map()
4      this.capacity = capacity
5  }
6
7  LRUCache.prototype.get = function (key) {
8      if (this.map.has(key)) {
9          let value = this.map.get(key)
10         // 重新set, 相当于更新到 map最后
11         this.map.delete(key)
12         this.map.set(key, value)
13         return value
14     }
15 }

```

```
16     return -1
17 }
18
19 LRUCache.prototype.put = function (key, value) {
20     // 如果有，就删了再赋值
21     if (this.map.has(key)) {
22         this.map.delete(key)
23     }
24
25     this.map.set(key, value)
26
27     // 判断是不是容量超了，淘汰机制
28     if (this.map.size > this.capacity) {
29         this.map.delete(this.map.keys().next().value)
30     }
31 }
```

4. 具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



5. 实际开发中遇到的最难的问题是什么 -- 教你如何带着答案找问题

此处以性能优化为例

常常进行前端性能优化的小伙伴们会发现，实际开发中性能优化总是阶段性的：页面加载很慢/卡顿 -> 性能优化 -> 堆叠需求 -> 加载慢/卡顿 -> 性能优化。

这是因为我们的项目往往也是阶段性的：快速功能开发 -> 出现性能问题 -> 优化性能 -> 快速功能开发。

建立一个完善的性能指标体系，便可以在需求开发阶段发现页面性能的下降，及时进行修复。

5.1 前端性能指标体系

为什么需要进行性能优化呢？这是因为一个快速响应的网页可以有效降低用户访问的跳出率，提升网页的留存率，从而收获更多的用户。参考《[经济时报](#)》[如何超越核心网页指标阈值，并使跳出率总体提高了 43%](#)，这个例子中主要优化了两个指标：Largest Contentful Paint (LCP) 和 Cumulative Layout Shift (CLS)。

除此之外，页面速度是一个重要的搜索引擎排名因素，它影响到你的网页是否能被更多用户访问。

常见的前端性能指标

我们来看下常见的前端性能指标，由于网页的响应速度往往包含很多方面（页面内容出现、用户可操作、流畅度等等），因此性能数据也由不同角度的指标组成：

- [First Contentful Paint \(FCP\)](#)：首次内容绘制，衡量从网页开始加载到网页任何部分呈现在屏幕上所用的时间
- [Largest Contentful Paint \(LCP\)](#)：最大内容绘制，衡量从网页开始加载到屏幕上渲染最大的文本块或图片元素所用的时间
- [First Input Delay \(FID\)](#)：首次输入延迟，衡量从用户首次与您的网站互动（点击链接、点按按钮或使用由 JavaScript 提供支持的自定义控件）到浏览器实际能够响应该互动的的时间
- [Interaction to Next Paint \(INP\)](#)：衡量与网页进行每次点按、点击或键盘交互的延迟时间，并根据互动次数选择该网页最差的互动延迟时间（或接近最高延迟时间）作为单个代表性值，以描述网页的整体响应速度
- [Time to Interactive \(TTI\)](#)：可交互时间，衡量的是从网页开始加载到视觉呈现、其初始脚本（若有）已加载且能够快速可靠地响应用户输入的时间
- [Total Blocking Time \(TBT\)](#)：总阻塞时间，测量 FCP 和 TTI 之间的总时间，在此期间，主线程处于屏蔽状态的时间够长，足以阻止输入响应
- [Cumulative Layout Shift \(CLS\)](#)：衡量从页面开始加载到其生命周期状态更改为隐藏之间发生的所有意外布局偏移的累计得分
- [Time to First Byte \(TTFB\)](#)：首字节时间，测量网络使用资源的第一个字节响应用户请求所需的时间

这些是 [User-centric performance metrics](#) 中介绍到的指标，其中 FCP、LCP、FID、INP/TTI 在我们常见的前端开发中会比较经常用到。

最简单的，一般前端应用都会关心以下几个指标：

1. FCP/LCP，该指标影响内容呈现给用户的体验，对页面跳出率影响最大。
2. FID/INP，该指标影响用户与网页交互的体验，对功能转化率和网页留存率影响较大。
3. TTI，该指标也为前端网页常用指标，页面可交互即用户可进行操作了。

除了这些简单的指标外，我们要如何建立起对网页完整的性能指标呢？一套成熟又完善的解决方案为 Google 的 [PageSpeed Insights \(PSI\)](#)。

PageSpeed Insights (PSI)

PageSpeed Insights (PSI) 是一项免费的 Google 服务，可报告网页在移动设备和桌面设备上的用户体验，并提供关于如何改进网页的建议。

PageSpeed Insights 和 Lighthouse 的区别主要为：

特征	PageSpeed Insights	Lighthouse
如何访问	https://pagespeed.web.dev/ （浏览器访问；无需登录）	Google Chrome 浏览器扩展 （推荐非开发人员使用） Chrome DevTools Node CLI 工具 Lighthouse CI
数据来源	Chrome 用户体验报告（真实数据） Lighthouse API（模拟实验室数据）	Lighthouse API
评估	一次一页	一次一页或一次多页
指标	核心网络生命、页面速度性能指标（首次内容绘制、速度指数、最大内容绘制、交互时间、总阻塞时间、累积布局偏移）	性能（包括页面速度指标）、可访问性、最佳实践、SEO、渐进式 Web 应用程序（如果适用）
建议	标有Opportunities and Diagnostics的部分提供了提高页面速度的具体建议。	标有Opportunities and Diagnostics的部分提供了提高页面速度的具体建议。堆栈包可用于定制改进建议。

简单来说，PageSpeed Insights 可同时获取实验室性能数据和用户实测数据，而 Lighthouse 则可获取实验室性能数据以及网页整体优化建议（包括但不限于性能建议）。

前端性能监控包括两种方式：合成监控（Synthetic Monitoring，SYN）、真实用户监控（Real User Monitoring，RUM）。这两种监控的性能数据，便是分别对应着实验室数据和用户实测数据。

实测数据是通过监控访问网页的所有用户，并针对其中每个用户的各自的体验，衡量一组给定的性能指标来确定的。和实验室数据不同，由于现场数据基于真实用户访问数据，因此它反映了用户的实际设备、网络条件和用户的地理位置。

当然，实测数据也可以由用户真实访问页面时进行上报收集，稍微大一点的前端应用都会这么做。但在此之前，如果你的前端网页没有做数据上报监控，也可以使用 PageSpeed Insights 工具进行简单的测试。但考虑到 PageSpeed Insights 收集的用户皆基于 Chrome 浏览器（CrUX），且需要登录的应用无法有效地获取真实数据，那么自行搭建一套性能指标体系则是最好的。

虽然实际上 PageSpeed Insights 服务并不能解决我们所有的问题，但是我们可以参考它的性能指标，来搭建自己的性能体系呀。

核心网页指标

参考 Google 的 [PageSpeed Insights](#)，我们知道 PSI 会报告真实用户在上一个 28 天收集期内的 First Contentful Paint (FCP)、First Input Delay (FID)、Largest Contentful Paint (LCP)、Cumulative Layout Shift (CLS) 和 Interaction to Next Paint (INP) 体验，同时 PSI 还报告了实验性指标首字节时间 (TTFB) 的体验。

其中，核心网页指标包括 FID/INP、LCP 和 CLS。

FID

[First Input Delay \(FID\)](#) 衡量的是从用户首次与网页互动（即，点击链接、点按按钮或使用由 JavaScript 提供支持的自定义控件）到浏览器能够实际开始处理事件处理脚本以响应该互动的的时间。

我们可以使用 [Event Timing API](#) 在 JavaScript 中衡量 FID：

```
1 new PerformanceObserver((entryList) => {for (const entry of entryList.getEntries()) {const delay =
  entry.processingStart - entry.startTime;
2   console.log('FID candidate:', delay, entry);}}}).observe({type: 'first-input', buffered: true});
```

实际上，从 2024 年 3 月开始，FID 将替换为 Interaction to Next Paint (INP)，后面我们会着重介绍。

LCP

[Largest Contentful Paint \(LCP\)](#) 指标会报告视口内可见的最大图片或文本块的呈现时间（相对于用户首次导航到页面的时间）。

我们可以使用 [Largest Contentful Paint API](#) 在 JavaScript 中测量 LCP:

```
1 new PerformanceObserver((entryList) => {for (const entry of entryList.getEntries()) {
2   console.log('LCP candidate:', entry.startTime, entry);}}).observe({type: 'largest-contentful-paint', buffered: true});
```

CLS

许多网站都面临布局不稳定的问题：DOM 元素由于内容异步加载而发生移动。

[Cumulative Layout Shift \(CLS\)](#) 指标便是用来衡量在网页的整个生命周期内发生的每次意外布局偏移的最大突发布局偏移分数。我们可以从 `Layout Instability` 方法中获得布局偏移：

```
1 addEventListener("load", () => {let DCLS = 0;new PerformanceObserver((list) => {
2   list.getEntries().forEach((entry) => {if (entry.hadRecentInput)return; // Ignore shifts after
   recent input.DCLS += entry.value;});}).observe({type: "layout-shift", buffered: true});});
```

布局偏移分数是该移动两个测量的乘积：影响比例和距离比例。

```
1 layout shift score = impact fraction * distance fraction
```

Interaction to Next Paint (INP)

FID 仅在用户首次与网页互动时报告响应情况。尽管第一印象很重要，但首次互动不一定代表网页生命周期内的所有互动。此外，FID 仅测量首次互动的“输入延迟”部分，即浏览器在开始处理互动之前必须等待的时间（由于主线程繁忙）。

[Interaction to Next Paint \(INP\)](#) 用于通过观察用户在访问网页期间发生的所有符合条件的互动的延迟时间，评估网页对用户互动的总体响应情况。

INP 不仅会衡量首次互动，还会考虑所有互动，并报告网页整个生命周期内最慢的互动。此外，INP 不仅会测量延迟部分，还会测量从互动开始，一直到事件处理脚本，再到浏览器能够绘制下一帧的完整时长。因此是 Interaction to Next Paint。这些实现细节使得 INP 能够比 FID 更全面地衡量用户感知的响应能力。

从 2024 年 3 月开始，INP 将替代 FID 加入 Largest Contentful Paint (LCP) 和 Cumulative Layout Shift (CLS)，作为三项稳定的核心网页指标。

INP 的计算方法是观察用户与网页进行的所有互动，而互动是指在同一逻辑用户手势触发的一组事件处理脚本。例如，触摸屏设备上的“点按”互动包括多个事件，如 `pointerup`、`pointerdown` 和 `click`。互动可由 JavaScript、CSS、内置浏览器控件（例如表单元素）或由以上各项驱动。

我们同样可以使用 [Event Timing API](#) 在 JavaScript 中衡量 FID：

```
1 new PerformanceObserver((entryList) => {for (const entry of entryList.getEntries()) {const delay =
   entry.processingStart - entry.startTime;}}).observe({type: 'event', buffered: true});
```

关于 INP 的优化，可以参考 [Optimize Interaction to Next Paint](#)。

[web-vitals JavaScript 库](#) 使用 `PerformanceObserver`，用于测量真实用户的所有 Web Vitals 指标，其方式准确匹配 Chrome 的测量方式，提供了上述提到的各种指标数据：CLS、FID、LCP、INP、FCP、TTFB。

我们可以使用 web-vitals 库来收集到所需的数据。

评估体验质量

PSI 根据网页指标计划设置了阈值，将用户体验质量分为三类：良好、需要改进或较差，具体可参考 [PageSpeed Insights 简介](#)。

值得注意的是，PSI 报告所有指标的第 75 百分位。

为便于开发者了解其网站上最令人沮丧的用户体验，选择第 75 百分位。通过应用上述相同阈值，这些字段指标值被归类为良好/需要改进/欠佳。

这与我们常见的前端性能指标监控不大一样，因为一般来说大家会取平均值来评估指标。而取 75 百分位这种方式，值得我们去好好思考哪种计算方式更能真实反应用户的体验。

当然，上述 PSI 的性能指标体系，也未必完全适合我们网页使用，我们还可以针对网页的实际情况做出调整。举个例子，网页的 FCP/LCP 虽然十分影响用户的留存，但如果是对于专注服务于老用户、操作频繁、使用时长长的应用来说，网页运行过程中的流畅性更值得关注。

5.2 常见性能优化方案

对于前端应用来说，网络耗时、页面加载耗时、脚本执行耗时、渲染耗时等耗时情况会影响用户的等待时长，而 CPU 占用、内存占用、本地缓存占用等则可能会导致页面卡顿甚至卡死。

因此，性能优化可以分别从耗时和资源占用两方面来解决，我个人也比较喜欢将其称为“时间”和“空间”两个维度。

时间角度优化：减少耗时

我们知道浏览器在页面加载过程中，会进行以下的步骤：

- 网络请求相关（发起 HTTP 请求从服务端获取页面资源，包括 HTML/CSS/JS/图片资源等）
- 浏览器解析 HTML 和渲染页面
- 加载 Javascript 代码时会暂停页面渲染（包括解析到外部资源，会发起 HTTP 请求获取并加载）

在浏览器的首次加载和渲染完成之后，不代表用户就可以马上交互和操作。根据业务代码加载过程，页面还会分别进入页面开始渲染、渲染完成、用户可交互等阶段。除此之外，页面交互过程中，会根据业务逻辑进行逻辑运算、页面更新。

题外话：为什么我们常常说要理解原理呢？性能优化便是个很好的例子，如果你不知道这个过程具体发生了什么，就很难找到地方下手去进行优化。

根据这个过程，我们可以从四个方面进行耗时优化：

1. 网络请求优化。
2. 首屏加载优化。
3. 渲染过程优化。
4. 计算/逻辑运行提速。

在前端性能优化实践中，网络请求优化和首屏加载优化方案使用频率最高，因为不管项目规模如何、各个模块和逻辑是否复杂，这两个方向的耗时优化方案都是比较通用的。相比之下，对于页面内容较多、交互逻辑/运算逻辑复杂的项目，才需要针对性地进行渲染过程优化和计算/逻辑运行提速。

一起来看看~

网络请求优化

网络请求优化的目标在于减少网络资源的请求和加载耗时，如果考虑 HTTP 请求过程，显然我们可以从几个角度来进行优化：

1. 请求链路：DNS 查询、部署 CDN 节点、缓存等。
2. 数据大小：代码大小、图片资源等。

对于请求链路，核心的方案常常包括使用缓存，比如 DNS 缓存、CDN 缓存、HTTP 缓存、后台缓存等等，前端的话还可以考虑使用 Service Worker、PWA 等技术。使用缓存并非万能药，很多使用由于缓存的存在，我们在功能更新修复的时候还需要考虑缓存的情况。除此之外，还可以考虑使用 HTTP/2、HTTP/3 等提升资源请求速度，以及对多个请求进行合并，减少通信次数；对请求进行域名拆分，提升并发请求数量。

数据大小则主要考对请求资源进行合理的拆分（CSS、Javascript 脚本、图片/音频/视频等）和压缩，减少请求资源的体积，比如使用 Tree-shaking、代码分割、移除用不上的依赖项等。

在请求资源返回后，浏览器会进行解析和加载，这个过程会影响页面的可见时间，通过对首屏加载的优化，可有效地提升用户体验。

首屏加载优化

首屏加载优化核心点在于两部分：

1. 将页面内容尽快地展示给用户，减少页面白屏时间。
2. 将用户可操作的时间尽量提前，避免用户无法操作的卡顿体验。

减少白屏时间除了我们常说的首屏加载耗时优化，还可以考虑使用一些过渡的动画，让用户感知到页面正在顺利加载，从而避免用户对于白屏页面或是静止页面产生烦躁和困惑。除了技术侧的优化，很多时候产品策略的调整，给用户带来的体验优化效果不低于技术手段优化，因此我们也需要重视。

[illegible]

- 对页面的内容进行分片/分屏加载
- 仅加载需要的资源，通过异步或是懒加载的方式加载剩余资源
- 使用骨架屏进行预渲染
- 使用差异化服务，比如读写分离，对于不同场景按需加载所需要的模块
- 使用服务端直出渲染，减少页面二次请求和渲染的耗时

有些时候，我们的页面也需要在客户端进行展示，此时可充分利用客户端的优势：

- 配合客户端进行资源预请求和预加载，比如使用预热 Web 容器
- 配合客户端将资源 and 数据进行离线，可用于下一次页面的快速渲染
- 使用秒看技术，通过生成预览图片的方式提前将页面内容提供给用户

除了首屏渲染以外，用户在浏览器页面过程中，也会触发页面的二次运算和渲染，此时需要进行渲染过程的优化。

渲染过程优化

渲染过程的优化要怎么定义呢？我们可以将其理解为首屏加载完成后，用户的操作交互触发的二次渲染。

主要思路是减少用户的操作等待时间，以及通过将页面渲染帧率保持在 60FPS 左右，提升页面交互和渲染的流畅度。包括但不限于以下方案：

- 使用资源预加载，提升空闲时间的资源利用率
- 减少/合并 DOM 操作，减少浏览器渲染过程中的计算耗时
- 使用离屏渲染，在页面不可见的地方提前进行渲染（比如 Canvas 离屏渲染）
- 通过合理使用浏览器 GPU 能力，提升浏览器渲染效率（比如使用 css transform 代替 Canvas 缩放绘制）

以上这些，是对常见的 Web 页面渲染优化方案。对于运算逻辑复杂、计算量较大的业务逻辑，我们还需要进行计算/逻辑运行的提速。

计算/逻辑运行提速

计算/逻辑运行速度优化的主要思路是“拆大为小、多路并行”，方式包括但不限于：

- 通过将 Javascript 大任务进行拆解，结合异步任务的管理，避免出现长时间计算导致页面卡顿的情况
- 将耗时长且非关键逻辑的计算拆离，比如使用 Web Worker
- 通过使用运行效率更高的方式，减少计算耗时，比如使用 Webassembly
- 通过将计算过程提前，减少计算等待时长，比如使用 AOT 技术
- 通过使用更优的算法或是存储结构，提升计算效率，比如 VSCode 使用红黑树优化文本缓冲区的计算
- 通过将计算结果缓存的方式，减少运算次数

以上便是时间维度的性能优化思路，还有空间维度的资源优化情况。

空间角度优化：降低资源占用

提到性能优化，大多数我们都在针对页面加载耗时进行优化，对资源占用的优化会更少，因为资源占用常常会直接受到用户设备性能和适应场景的影响，大多数情况下优化效果会比耗时优化局限，因此这里也只能说一些大概的思路。

资源占用常见的优化方式包括：

- 合理使用缓存，不滥用用户的缓存资源（比如浏览器缓存、IndexedDB），及时进行缓存清理
- 避免存在内存泄露，比如尽量避免全局变量的使用、及时解除引用等
- 避免复杂/异常的递归调用，导致调用栈的溢出
- 通过使用数据结构享元的方式，减少对象的创建，从而减少内存占用

说到底，我们在做性能优化的时候，其实很多情况下会依赖时间换空间、空间换时间等方式。性能优化没有银弹，只能根据自己项目的实际情况做出取舍，选择相对合适的一种方案去进行优化。

对于页面耗时和资源占用的性能优化分析，大部分情况都可以使用 Chrome 开发者工具进行针对性的分析和优化

5.3 加载流程分析

其实我们在性能优化的归纳篇有简单说过，页面加载的过程其实跟我们常常提起的浏览器页面渲染流程几乎一致：

1. 网络请求，服务端返回 HTML 内容。
2. 浏览器一边解析 HTML，一边进行页面渲染。
3. 解析到外部资源，会发起 HTTP 请求获取，加载 Javascript 代码时会暂停页面渲染。
4. 根据业务代码加载过程，会分别进入页面开始渲染、渲染完成、用户可交互等阶段。
5. 页面交互过程中，会根据业务逻辑进行逻辑运算、页面更新。

那么，我们可以针对其中的每个步骤做优化，主要包括：资源获取、资源加载、页面可见、页面可交互。

资源获取

资源获取主要可以围绕两个角度做优化：

- 资源大小
- 资源缓存

资源大小

一般来说，前端都会在打包的时候做资源大小的优化，资源类型包括 HTML、JavaScript、CSS、图片等。优化的方向包括：

(1) 合理的对资源进行分包。

首次渲染时只保留当前页面渲染需要的资源，将可以异步加载、延迟加载的资源拆离。通常我们会在代码编译打包的时候做处理，比如使用 Webpack 将代码拆到不同的 bundle 包中。

(2) 移除不需要的代码。

我们项目中常常会引入许多开源代码，同时我们自己也会实现很多的工具方法，但是实际上并不是全部相关的代码都是最终需要执行的代码，所以我们可以打包的时候移除不需要的代码。现在基本大多数的打包工具都提供了类似的能力，比如 Tree-shaking。除此之外，如果我们的项目较大，使用和依赖了多个不同的仓库。如果在不同的代码仓库里，都依赖了同样的 npm 代码包，那么我们可能会遇到打包时引入多次同样的 npm 包的情况。一般来说，我们在管理依赖包的时候，可以使用 `peerDependency` 来进行管理，避免多次安装依赖、以及版本不一致导致的多次打包和安装等情况。

(3) 资源压缩和合并。

代码压缩也常常是在打包阶段进行的，包括 JavaScript 和 CSS 等代码，在一些情况下也可以使用图片合并（雪碧图的生成）。通常也是使用的打包工具以及插件自带的压缩能力，开启压缩后的代码可能比较难定位，可以配合 Source Mapping 来进行问题定位。

除了打包时的压缩，我们在页面加载的时候也可以启用 HTTP 的 gzip 压缩，可以减少资源 HTTP 请求的耗时。

资源缓存

资源缓存的优化，其实更多时候跟我们的资源获取的链路有关，包括：

- 减少 DNS 查询时间，比如使用浏览器 DNS 缓存、计算机 DNS 缓存、服务器 DNS 缓存
- 合理地使用 CDN 资源，有效地减少网络请求耗时
- 对请求资源进行缓存，包括但不限于使用浏览器缓存、HTTP 缓存、后台缓存，比如使用 Service Worker、PWA 等技术

其实，我们观察资源获取的链路，获取除了大小和缓存的角度以外，还可以做更多的优化，比如：

- 使用 HTTP/2、HTTP/3，提升资源请求速度
- 对请求进行优化，比如对多个请求进行合并，减少通信次数
- 对请求进行域名拆分，提升并发请求数量

资源加载

资源加载步骤中，我们一般也有以下的优化角度：

- 加载流程拆分
- 资源懒加载
- 资源预加载

加载流程拆分

页面的加载过程，常常分为两个阶段：页面可见、页面可交互。

前面我们讲了对资源做拆分，在页面启动加载的时候仅加载需要的资源，拆分的过程则可以结合上述的两个阶段来做处理。

(1) 页面可见。

页面可见可以分为部分可见以及内容完全可见。

对于部分可见，一般来说可以做 loading 的展示或是直出，让用户知道页面正在加载中，而非无响应。

对于内容完全可见，则是用户可视区域内的内容完全渲染完毕。除此之外，当前可视范围以外的内容，则可以拆离出首屏的分包，通过预加载或是懒加载的方式进行异步加载。

(2) 页面可交互。

同样的，页面可交互也可以分为部分可交互以及完全可交互。

一般来说，组件的样式渲染仅需要 HTML 和 CSS 加载完成即可，而组件的功能则可能需要加载具体的功能代码。对于复杂或是依赖资源较多的功能，加载的耗时可能相对较长。在这样的情况下，我们可以选择将该部分的资源做异步加载。

在初始的内容加载完毕之后，剩下的资源需要延迟加载。对于页面功能完全可交互，同样依赖于分包资源延迟加载。加载流程的优化，不管是页面可见，还是页面可交互，都离不开延迟加载。

延迟加载可分为两种方式进行加载：懒加载和预加载。因此，资源懒加载和预加载也是加载流程中很重要的一部分。

资源懒加载

我们常说的懒加载其实又被称为按需加载，顾名思义就是需要用到时候才会进行加载。通过将非必要功能进行懒加载的方式，可以有效地减少页面的初始加载速度，提升页面加载的性能。

常见的场景比如某些组件在渲染时不具备完整的功能，当用户点击的时候，才进行对应逻辑的获取和加载。遇到点击时未加载完成的情况下，可以通过适当的方式提示用户功能正在加载中。

资源懒加载常常也是跟资源分包一起进行，大多数前端框架（比如 Vue、React、Angular）也都提供了懒加载的能力，也可以[配合 Webpack 打包](#) 做处理。

资源预加载

资源预加载也称为闲时加载，很多时候我们可以在页面空闲的时候，对一些用户可能会用到的资源做提前加载，以加快后续渲染或者操作的时间。

仔细一看，资源预加载和资源懒加载都比较相似，都会通过将资源拆离的方式做成异步延迟的方式加载。两者的区别在于：

- 懒加载的功能只会在需要的时候才进行加载，因为一些功能用户可能不会使用到，比如帮助中心、反馈功能等等
- 预加载的功能则是在不阻塞核心功能的时候，尽可能利用空闲的资源提前加载，这部分的功能则是用户很可能会使用到，比如获取下一屏页面的内容数据

复杂场景下的加载流程

在页面到达可交互状态之后，后续的加载流程也可以根据业务场景做后续的优化。对于一些复杂的业务，我们可以结合业务的特点做更进一步的性能优化。

复杂加载流程管理

对于页面初始化流程过于复杂的应用来说，我们可以对加载流程做任务的拆分，分阶段地进行加载。

举个例子，假设我们需要在 Web 端加载 VSCode，那么我们可能需要考虑以下各个功能的加载：

- 整体页面框架
 - 顶部菜单栏
 - 左侧工具栏
 - 底部状态栏
 - 文件目录栏
 - 文件详情
 - 搜索功能
 - 插件功能

以上只是我按照自己想法粗略拆分的功能，我们可以简单分成几个加载阶段：

1. 页面整体框架加载完成。此时可以看到各个功能区域的分布，包括顶部菜单栏、左侧工具栏、底部状态栏、项目内容区域等等，但这些区域的内容未必都完全加载完成。
2. 通用功能加载完成。比如顶部菜单栏、左侧工具栏、底部状态栏等等，一些具体的菜单或是工具的功能可以做按需加载和预加载，比如搜索功能。
3. 项目内容相关框架加载完成。此时可以看到项目相关的内容区域，比如文件目录、当前文件的内容详情等等。
4. 插件功能。用户安装的插件，在核心功能都加载完成之后再获取和加载。

当我们根据项目的具体加载过程做了阶段划分之后，则可以将我们的代码做任务拆分，可以拆分成串行和并行的任务。串行的任务比如按照阶段划分的大任务，并行的任务则可以是某个阶段内的小任务，其中也可以包括一些异步执行的任务，或是延迟加载的任务。

长耗时任务的拆离

如果我们的应用中会有耗时较长的计算任务，比如拉取回来的数据需要计算处理后才能渲染，那么我们可以对这些耗时较长的任务做任务拆分。

同样的，我们还是回到 Web 端加载 VsCode 的场景。假设我们在加载某个特别大的文件，则可以考虑分别对该文件的内容获取、数据转换做任务拆分，比如分片获取该文件的内容，根据分片的内容做渲染的计算，计算过程如果耗时较长，也可以做异步任务的拆分，甚至可以结合 Web Worker 和 WebAssembly 等技术做更多的优化。

读写分离

对于交互复杂、需要加载的资源较多的情况下，如果用户的权限只是可读，那么对于编辑相关的功能可以做资源拆离，对于有权限的用户才进行编辑能力的加载。

读写分离其实属于资源拆分的一种具体场景，我们可以结合业务的具体场景做具体的功能拆分，比如管理员权限相关的管理功能，也是类似的优化场景

5.4 卡顿检测

首先，我们来看看可以怎么主动检测卡顿的出现。

卡顿，顾名思义则是代码执行产生长耗时，导致浏览器无法及时响应用户的操作。那么，我们可以基于不同的方案，来监测当前页面响应的延迟。

Worker 心跳方案

对应浏览器来说，由于 JavaScript 是单线程的设计，当卡顿发生的时候，往往是由于 JavaScript 在执行过长的逻辑，常见于大量数据的遍历操作，甚至是进入死循环。

利用这个特效，我们可以在页面打开的时候，就启动一个 Worker 线程，使用心跳的方式与主线程进行同步。假设我们希望能监测 1s 以上的卡顿，我们可以设置主线程每间隔 1s 向 Worker 发送心跳消息。（当然，线程通讯本身需要一些耗时，且 JavaScript 的计时器也未必是准时的，因此心跳需要给予一定的冗余范围）

由于页面发生卡顿的时候，主线程往往是忙碌状态，我们可以通过 Worker 里丢失心跳的时候进行上报，就能及时发现卡顿的产生。

但是其实 Worker 更多时候用于检测网页崩溃，用来检测卡顿的效果其实还不如使用 `window.requestAnimationFrame`，因为线程通信的耗时和延迟导致该方案不大准确。

`window.requestAnimationFrame` 方案

有简单提到一些卡顿的检测方案，市面上大多数的方案也是基于 `window.requestAnimationFrame` 方法来检测是否有卡顿出现。

`window.requestAnimationFrame()` 会在浏览器下次重绘之前调用，常常用来更新动画。这是因为 `setTimeout` / `setInterval` 计时器只能保证将回调添加至浏览器的回调队列(宏任务)的时间，不能保证回调队列的运行时间，因此使用 `window.requestAnimationFrame` 会更合适。

通常来说，大多数电脑显示器的刷新频率是 60Hz，也就是说每秒钟 `window.requestAnimationFrame` 会被执行 60 次。因此可以使用 `window.requestAnimationFrame` 来监控卡顿，具体的方案会依赖于我们项目的要求。

比如，有些人会认为连续出现 3 个低于 20 的 FPS 即可认为网页存在卡顿，这种情况下我们则针对这个数值进行上报。

除此之外，假设我们认为页面中存在超过特定时间（比如 1s）的长耗时任务即存在明显卡顿，则我们可以判断两次 `window.requestAnimationFrame` 执行间超过一定时间，则发生了卡顿。

使用 `window.requestAnimationFrame` 监测卡顿需要注意的是，他是一个被十分频繁执行的代码，不应该处理过多的逻辑。

Long Tasks API 方案

熟悉前端性能优化的开发都知道，阻塞主线程达 50 毫秒或以上的任务会导致以下问题：

- 可交互时间（TTI）延迟

- 严重不稳定的交互行为 (轻击、单击、滚动、滚轮等) 延迟
- 严重不稳定的事件回调延迟
- 紊乱的动画和滚动

因此，W3C 推出 [Long Tasks API](#)。长任务 (Long task) 定义了什么连续不间断的且主 UI 线程繁忙 50 毫秒及以上的时间区间。比如以下常规场景：

- 长耗时的事件回调
- 代价高昂的回流和其他重绘
- 浏览器在超过 50 毫秒的事件循环的相邻循环之间所做的工作

参考 [Long Tasks API -- MDN](#)

我们可以使用 `PerformanceObserver` 这样简单地获取到长任务：

```
1 var observer = new PerformanceObserver(function (list) {var perfEntries = list.getEntries();for (var i = 0; i < perfEntries.length; i++) {// 分析和上报关键卡顿信息}});// 注册长任务的观察
2 observer.observe({ entryTypes: ["longtask"] });
```

相比 `requestAnimationFrame`，使用 Long Tasks API 可避免调用过于频繁的问题，并且 `performance timeline` 的任务优先级较低，会尽可能在空闲时进行，可避免影响页面其他任务的执行。但需要注意的是，该 API 还处于实验性阶段，兼容性还有待完善，而我们卡顿常常发生在版本较落后、性能较差的机器上，因此兜底方案也是十分需要的。

PerformanceObserver 卡顿检测

前面也提到，卡顿产生于用户操作后网页无法及时响应。根据这个原理，我们可以使用 `PerformanceObserver` 监听用户操作，检测是否产生卡顿：

```
1 new PerformanceObserver((list) => {
2   list.getEntries().forEach((entry) => {const duration = entry.duration;const delay =
    entry.processingStart - entry.startTime;const eventHandlerTime = entry.processingEnd -
    entry.processingStart;
3     console.log(`Total duration: ${duration}`);
4     console.log(`Event delay: ${delay}`);
5     console.log(`Event handler duration: ${eventHandlerTime}`);});}).observe({ type: "event" });
```

这种方式的好处是避免频繁在 `requestAnimationFrame` 中执行任务，这也是官方鼓励开发者使用的方式，它避免了轮询，且被设计为低优先级任务，甚至可以从缓存中取出过往数据。

但该方式仅能发现卡顿，至于具体的定位还是得配合埋点和心跳进行会更有效。

卡顿埋点上报

不管是哪种卡顿监控方式，我们使用检测卡顿的方案发现了卡顿之后，需要将卡顿进行上报才能及时发现问题。但如果我们仅仅上报了卡顿的发生，是不足以定位和解决问题的。

卡顿打点

那么，我们可以通过打点的方式来大概获取卡顿发生的位置。

举个例子，假设我们一个网页中，关键的点和容易产生长耗时的操作包括：

1. 加载数据。
2. 计算。

3. 渲染。
4. 批量操作。
5. 数据提交。

那么，我们可以在这些操作的地方进行打点。假设我们卡顿工具的能力主要有两个：

```
1 interface IJank {
2   _jankLogs: Array<IJankLogInfo & { logTime: number }>; // 打点log(jankLogInfo: IJankLogInfo): void; // 心跳_heartbeat(): void;}
```

那么，当我们在页面加载的时候分别进行打点，我们的堆栈可能是这样的：

```
1 _jankLogs = [{
2   module: "数据层",
3   action: "加载数据",
4   logTime: xxxxx,},{
5   module: "渲染层",
6   action: "计算",
7   logTime: xxxxx,},{
8   module: "渲染层",
9   action: "渲染",
10  logTime: xxxxx,},{
11  module: "数据层",
12  action: "批量操作计算",
13  logTime: xxxxx,},{
14  module: "数据层",
15  action: "数据提交",
16  logTime: xxxxx,},];
```

当卡顿心跳发现卡顿产生时，我们可以拿到堆栈的数据，比如当用户在批量操作之后发生卡顿，假设此时我们拿到堆栈：

```
1 _jankLogs = [{
2   module: "数据层",
3   action: "加载数据",
4   logTime: xxxxx,},{
5   module: "渲染层",
6   action: "计算",
7   logTime: xxxxx,},{
8   module: "渲染层",
9   action: "渲染",
10  logTime: xxxxx,},{
11  module: "数据层",
12  action: "批量操作计算",
13  logTime: xxxxx,},];
```

这意味着卡顿发生时，最后一次操作是 `数据层--批量操作计算`，则我们可以认为是该操作产生了卡顿。

我们可以将 `module` / `action` 以及具体的卡顿耗时一起上报，这样就方便我们监控用户的大盘卡顿数据了，也较容易地定位到具体卡顿产生的位置。

心跳打点

当然，上述方案如果能达到最优效果，则我们需要在代码中关键的位置进行打点，常见的比如数据加载、计算、事件触发、JavaScript 加载等。

我们可以将打点方法做成装饰器，自动给 `class` 中的方法进行打点。如果埋点数据过少，可能会产生误报，那么我们可以增加心跳的打点：

```
1  IJank._heartbeat = () => {
2    IJank.log({
3      module: "Jank",
4      action: "heartbeat",
5      logTime: xxxxx,});};
```

当我们心跳产生的时候，会更新堆栈数据。假设发生卡顿的时候，我们拿到这样的堆栈信息：

```
1  _jankLogs = [{
2    module: "数据层",
3    action: "加载数据",
4    logTime: xxxxx,},{
5    module: "Jank",
6    action: "heartbeat",
7    logTime: xxxxx,},{
8    module: "Jank",
9    action: "heartbeat",
10   logTime: xxxxx,},{
11   module: "渲染层",
12   action: "计算",
13   logTime: xxxxx,},{
14   module: "Jank",
15   action: "heartbeat",
16   logTime: xxxxx,},{
17   module: "渲染层",
18   action: "渲染",
19   logTime: xxxxx,},{
20   module: "Jank",
21   action: "heartbeat",
22   logTime: xxxxx,},{
23   module: "数据层",
24   action: "批量操作计算",
25   logTime: xxxxx,},{
26   module: "Jank",
27   action: "heartbeat",
28   logTime: xxxxx,},];
```

显然，卡顿发生时最后一次打点为 `Jank--heartbeat`，这意味着卡顿并不是产生于 `数据层---批量操作计算`，而是产生于该逻辑后的一个不知名逻辑。在这种情况下，我们可能还需要再在可疑的地方增加打点，再继续观察。

JavaScript 加载打点

有一个用于监控一些懒加载的 JavaScript 代码的小技巧，我们可以使用 `PerformanceObserver` 获取到 JavaScript 代码资源拉取回来后的时机，然后进行打点：

```
1  performanceObserver = new PerformanceObserver((resource) => {const entries = resource.getEntries();
2    entries.forEach((entry: PerformanceResourceTiming) => {// 获取 JavaScript 资源if (entry.initiatorType
3      != "script") return;// 打点this.log({
4      moduleValue: "compileScript",
5      actionValue: entry.name,});});});// 监测 resource 资源
performanceObserver.observe({ entryTypes: ["resource"] });
```


当卡顿产生时，堆栈的最后一个日志如果为 `compileScript--bundle_xxxx` 之类的，则可以认为该 JavaScript 资源在加载的时候耗时较长，导致卡顿的产生。

通过这样的方式，我们可以有效监控用户卡顿的发生，以及卡顿产生较多的逻辑，然后进行相应的问题定位和优化

6. 面试要求的架构，工程化到底是什么--如何培养自己的架构思维

6.1 大型前端项目常见的问题与解决思路

或许你会感到疑惑，怎样的项目算是大型前端项目呢？我自己的理解是，项目的开发人员数量较多（10 人以上？）、项目模块数量/代码量较多的项目，都可以理解为大型前端项目了。

在前端业务领域中，除了大型开源项目（热门框架、VsCode、Atom 等）以外，协同编辑类应用（比如在线文档）、复杂交互类应用（比如大型游戏）等，都可以称得上是大型前端项目。对于这样的大型前端项目，我们在开发中常常遇到的问题包括：

1. 项目代码量大，不管是编译、构建，还是浏览器加载，耗时都较多、性能也较差。
2. 各个模块间耦合严重，功能开发、技术优化、重构工作等均难以开展。
3. 项目交互逻辑复杂，问题定位、BUG 修复等过程效率很低，需要耗费不少精力。
4. 项目规模太大，每个人只了解其中一部分，需求改动到不熟悉的模块时常常出问题。

其实大家也能看到，大型前端项目中主要的问题便是“管理混乱”。所以我个人觉得，对于代码管理得很混乱的项目，你也可以认为是“大型”前端项目（笑）。

问题 1：项目代码量过大

对于代码量过大（比如高达 30W 行）的项目，如果不做任何优化直接全量跑在浏览器中，不管是加载耗时增加导致用户等待时间过长，还是内存占用过高导致用户交互卡顿，都会给用户带来不好的体验。

其中，对于代码量、文件过多这样的性能优化，可以总结为两个字：

- 拆：拆模块、拆公共库、拆组件库
- 分：分流程、分步骤

项目代码量过大不仅仅会影响用户体验，对于开发来说，代码开发过程中同样存在糟糕的体验：由于代码量过大，开发的本地构建、编译都变得很慢，甚至去打水 + 上厕所回来之后，代码还没编译完。

从维护角度来看，一个项目的代码量过大，对开发、编译、构建、部署、发布流程都会同样带来不少的压力。因此除了浏览器加载过程中的代码拆分，对项目代码也可以进行拆分，一般来说有两种方式：

5. multirepo，多仓库模块管理，通过工作流从各个仓库拉取代码并进行编译、打包。

- 优点：模块可根据需要灵活选择各自的编译、构建工具；每个仓库的代码量较小，方便维护
- 缺点：项目代码分散在各个仓库，问题定位困难（使用 `npm link` 有奇效）；模块变动后，需要更新相关仓库的依赖配置（使用一致的版本控制和管理方式可减少这样的问题）

6. monorepo，单仓库模块管理，可使用 lerna 进行包管理。

- 优点：项目代码可集中进行管理，使用统一的构建工具；模块间调试方便、问题定位和修复相对容易
- 缺点：仓库体积大，对构建工具和机器性能要求较高；对项目文件结构和管理、代码可测试和维护性要求较高；为了保证代码质量，对版本控制和 Git 工作流要求更高

两种包管理模式各有优劣，一般来说一个项目只会采用其中一种，但也可以根据具体需要进行调整，比如统一的 UI 组件库进行分仓库管理、核心业务逻辑在主仓库内进行拆包管理。

题外话：很多人常常在争论到底是单仓好还是多仓好，个人认为只要能解决开发实际痛点的仓，都是好仓，有时候过多的理论也需要实践来验证。

问题 2：模块耦合严重

不同的模块需要进行分工和配合，因此相互之间必然会产生耦合。在大型项目中，由于模块数量很多（很多时候也是因为代码量过多），常常会遇到模块耦合过于严重的问题：

1. 模块职责和边界定义不清晰，导致模糊的工作可能存在多个模块内。
2. 各个模块没有统一管理，导致模块在状态变更时需要手动通知相关模块。
3. 模块间的通信方式设计不合理，导致全局事件满天飞、A 模块内直接调用 B 模块等问题，隐藏的引用和事件可能导致内存泄露。

对于模块耦合严重的模块，常见的解耦方案比如：

- 使用事件驱动的方式，通过事件来进行模块间通信
- 使用依赖倒置进行依赖解耦

事件驱动进行模块解耦

使用事件驱动的方式，可以快速又简单地实现模块间的解耦，但它常常又带来了更多的问题，比如：

- 全局事件满天飞，不知道某个事件来自哪里，被多少地方监听了
- 无法进行事件订阅的销毁管理，容易存在内存泄露的问题
- 事件维护困难，增加和调整参数影响面广，容易触发 bug

依赖倒置进行模块解耦

我们还可以使用依赖倒置进行依赖解耦。依赖倒置原则有两个，包括：

1. 高层次的模块不应该依赖于低层次的模块，两者都应该依赖于抽象接口。
2. 抽象接口不应该依赖于具体实现，而具体实现则应该依赖于抽象接口。

使用以上方式进行设计的模块，不会依赖具体的模块和细节，只按照约定依赖抽象的接口。

如果项目中有完善的依赖注入框架，则可以使用项目中的依赖注入体系，像 Angular 框架便自带依赖注入体系。依赖注入在大型项目中比较常见，对于各个模块间的依赖关系管理很实用，比如 VsCode 中就有使用到依赖注入。

问题 3：问题定位效率低

在对模块进行拆分和解耦、使用了模块负责人机制、进行包拆分管理之后，虽然开发同学可以更加专注于自身负责模块的开发和维护，但有些时候依然无法避免地要接触到其它模块。

对于这样大型的项目，维护过程（熟悉代码、定位问题、性能优化等）由于代码量太多、各个函数的调用链路太长，以及函数执行情况黑盒等问题，导致问题定位异常困难。要是遇到代码稍微复杂点，比如事件反复横跳的，即使使用断点调试也能看到眼花，蒸汽眼罩都得多买一些（真的贵啊）。

对于这些问题，其实可以有两个优化方式：

1. 维护模块指引文档，方便新人熟悉现有逻辑。文档主要介绍每个模块的职责、设计、相关需求，以及如何调试、场景的坑等。
2. 尝试将问题定位过程进行自动化实现，比如模块负责人对自身模块执行的关键点进行标记（使用日志或者特定的断点工具），其他开发可根据日志或是通过开启断点的方式来直接定位问题

这个过程，其实是将模块负责人的知识通过工具的方式授予其他开发，大家可以快速找到某个模块经常出问题的地方、模块执行的关键点，根据建议和提示进行问题定位，可极大地提升问题定位的效率。

除了问题定位以外，各个模块和函数的调用关系、调用耗时也可以作为系统功能和性能是否有异常的参考。

因此，我们还可以通过将调用堆栈收集过程自动化、接入流水线，在每次发布前合入代码时执行相关的任务，对比以往的数据进行分析，生成系统性能和功能的风险报告，提前在发布前发现风险。

问题 4：项目复杂熟悉成本过高

即使在项目代码量大、项目模块过多、耦合严重的情况下，项目还在不断地进行迭代和优化。遇到这样的项目，基本上没有一个人能熟悉所有模块的所有细节，这会带来一些问题：

- 对于新需求、新功能，开发无法完整地评估技术方案是否可以实现、会不会带来新的问题
- 需求开发时需要改动不熟悉的代码，无法评估是否存在风险
- 架构级别的优化工作，难以确定是否可以真正落地
- 一些模块遗留的历史债务，由于工作进行过多次交接，相关逻辑已无人熟悉，无法进行处理

导致这些问题的根本原因有两个：

- 开发无法专注于某个模块开发
- 同一个模块可能被多个人调整和变更

对于这种情况，可以使用模块负责人的机制来对模块进行所有权分配，进行管理和维护：

1. 每个开发都认领（或分配）一个或多个模块，并要求完全熟悉和掌握模块的细节，且维护文档进行说明。
2. 对于需求开发、BUG 修复、技术优化过程中涉及到非自身的模块，需要找到对应模块的负责人进行风险评估和代码 Review。
3. 模块的负责人负责自身模块的技术优化方案，包括性能优化、自动化测试覆盖、代码规范调整等工作。
4. 对于较核心/复杂的模块，可由多个负责人共同维护，协商技术细节。

通过模块负责人机制，每个模块都有了对应的开发进行维护和优化，开发也可以专注于自身的某些模块进行功能开发。在人员离职和工作内容交接的时候，也可以通过文档 + 负责人权限的方式进行模块交接。

6.2 我所理解的前端工程化

相信大家的日常工作中也能感受到，相比于从 0 到 1 搭建项目，我们的大部分工作都是在维护项目，基于现有的项目上进行迭代和开发。正所谓铁打的营盘流水的兵，每个项目都会经历很多开发的参与、协作和交接，在这个过程中常常会遇到很多的问题，这些问题可以分为两类：系统质量的下降，以及开发效率的下降。

系统质量的下降

BUG 的出现有很多的可能性，比如需求设计不严谨、代码实现的逻辑有漏洞、不在预期之内的异常逻辑分支，等等。除了方案设计和思考的经验不足，BUG 很多时候也会因为对项目的不熟悉、对系统的理解不深入引入，这意味着以下的过程会导致 BUG 的增加：

1. 项目频繁地调整（新增或者更换）开发人员，由于不熟悉项目，每个新加入的小伙伴都可能会埋下新的 BUG。
2. 系统功能新增和迭代、不断壮大，各个模块间的耦合增加、复杂度增加。如果没法掌握系统的所有细节，很可能牵一发而动全身，产生自己认知以外的 BUG。

对于处于快速迭代、不断拓展阶段的项目来说，不管是人员的变动、还是项目的拓展都是无法避免的。除此之外，为了降低系统的复杂度，当项目发展到一定阶段的时候，会对系统进行局部或是整体的架构调整，比如模块的拆分、各个模块间的依赖解耦、引入新的状态管理工具、重复逻辑进行抽象和封装等等。

新技术的引入会缓解系统复杂度带来的稳定性问题，但同时也可能会引入新的问题，比如：

- 部分功能无法与新技术兼容，成为历史遗留问题
- 较大范围的架构调整影响面很广，可能埋下难以发现的 BUG

可见，一个项目不断发展的过程中，都会面临系统质量下降的问题。

提升系统质量

为了提升系统质量，我们需要对项目进行合理的架构调整，提升系统的可读性、可维护性、可测试性、稳定性，从而提升系统发布的稳定性。

我们在进行架构设计时，需要根据项目的预期和现状来设计，保留拓展性的同时，避免过度设计。因此，随着项目不断发展，原有的架构设计可能不再适合，此时我们需要进行优化和调整，比如：

- 引入新的技术和工具
- 团队成员增加，沟通成本和对规范的理解出现差异
- 项目代码量和文件数的增加

- 进行自动化测试能力的覆盖
- 搭建完善的监控和告警体系

在这个过程中，我们可能分别引入了新的代码构建、代码规范和自动化测试工具，搭建了新的监控系统、发布系统、流程控制等，这些都属于前端工程化的一部分。

但是对于开发来说，开发流程变得繁琐，意味着工作内容更复杂，同时还增加了很多新工具和系统的熟悉成本。那么，我们还可以通过优化项目的研发和发布流程，来提升项目的开发效率。

开发效率的下降

系统上线之后，开发的工作内容重心，会从功能开发逐渐转向其它内容。除了新功能的评审和设计以外，还会包括：

- 用户反馈问题跟进和定位
- 线上 BUG 修复和紧急发布
- 处理系统的监控告警，排查异常问题
- 新功能灰度发布过程，自测、产品验证功能、提测、修复 BUG、灰度发布等各个流程都需要人工操作和主动关注
- 为了保证系统质量，需要完善自动化测试能力，包括单元测试、UI 测试、集成测试等
- 项目成员的调整，需要进行工作的交接、指导对方的工作内容等

开发的工作内容变得复杂，需要关注的事情也更多，对于各个系统（监控告警系统、日志系统、测试系统、发布系统等）也都需要熟悉成本和操作成本。在各个工作内容之间切换，也常常容易出现步骤的遗漏，导致一些流程上的问题，比如：

1. 系统灰度到一半，处理其它事情忘了全量。
2. 系统发布之后，去处理紧急 BUG、忘记看监控，直到收到大量的用户反馈。
3. 线上紧急 BUG 修复了，急着发布忘了进行自动化测试。

随着项目规模变大，系统的复杂度也随之上升，上面所提到的工作量也都会增加，开发效率会肉眼可见地受到影响。以前一天工作量的功能开发，如今需要三天时间才能完成，因为每天只有三分之一的时间（甚至更少）可以用来开发新功能。

在这个项目阶段，开发每天的杂事太多、效率太低、浑浑噩噩不知道都做了些什么，团队面临着项目复杂度上升、系统质量不稳定、技术债务越来越多、团队工作效率下降等问题。

提升开发效率

项目研发和发布流程优化的核心点在于：将一切需要手动操作和关注的内容自动化。

那么，我们先来梳理下项目开发和发布过程中，到底有多少繁琐的工作可以进行自动化。一般来说，开发在接到产品需求单后，会涉及到分支管理、代码构建、环境部署、测试/验证、问题修复、灰度发布、监控告警、需求单状态扭转等各个流程。

每一次功能发布，都需要花费很多的精力在各个流程步骤上。我们可以将这些步骤都转为自动化，就可以让开发的精力聚焦在功能的设计和实现上。对于流程自动化，业界比较成熟的解决方案是使用持续集成（continuous integration，简称 CI）和持续部署（continuous deployment，简称 CD）：

- 持续集成（CI）：目的是让产品可以快速迭代，同时还能保持高质量
- 持续部署（CD）：目的是代码在任何时刻都是可部署、可进入生产阶段

CI 在项目中的实践，指团队成员频繁（一天多次）地提交代码到主干分支，每次提交后会自动触发自动化验证的任务集合，以便尽可能地提前发现问题；CD 在项目中的实践，指代码通过评审以后，可自动化、可控、多批次地部署到生产环境，CD 的前提是能自动化完成测试、构建、部署等步骤。

CI/CD 在项目中的落地，很多时候会表现为流水线的开发模式：通过建立完整的 CI/CD 流水线，涵盖整个研发流程，可有效地提高效率。一般来说，我们可以搭建这样的 CI/CD 流水线：

- 需求单分配：分配并自动拉取相应 Git 分支
- 代码提交：代码规范检查 + 自动化测试 + 部署测试环境 + 根据需求单配置通知相应的产品和测试
- 产品验证/功能测试：BUG 单自动关联需求单和分支 + 验证完成后，根据需求单通知开发侧

- BUG 修复：代码规范检查 + 自动化测试 + 部署测试环境 + 根据分支 BUG 单和需求单通知测试 + 验证完成后，根据需求单通知开发侧
- 代码合入主干：向团队成员发起代码 Review + Review 通过后代码合入主干
- 日常发布：定时器发起发布流程 + 预发布环境部署 + 进行自动化测试 + 测试通过后进入灰度过程
- 灰度发布：根据配置比例进行灰度 + 灰度过程中自动化进行监控 + 可选择性进入快速回滚流程
- 全量发布：自动扭转需求单状态，并将版本进行归档（Git Tag）

通过将以上流程自动化，可以节省开发的很多人工操作时间、提升开发效率的同时，也避免了人工操作容易出现的遗漏和失误。将自动化流水线与通知/告警机器人、工作群、需求单系统、BUG 系统、代码管理系统、发布系统、监控系统结合，实现全研发和发布流程的自动化，开发可从各种杂事中释放，专注于功能开发的实现。

越是大规模、系统建设完备的团队，开发流程中消耗在多人协作和各个系统的操作中的精力越多，搭建 CI/CD 后更能体会到自动化流程带来的便利。

当然，搭建 CI/CD 的过程中，也需要投入不少的人力精力。因此，很多时候我们可以考虑性价比，从对研发效能影响最大的痛点开始进行建设，可以最快速和有效地提升团队的开发效率，让更多的人愿意参与到 CI/CD 的建设中

6.3 依赖与解耦

既然要研究怎么让模块解耦，那当然要从根源来分析：依赖它到底从何而来？

依赖其实是在我们想把代码写好的那一刻开始产生的。为什么这么说呢？因为大多数代码都是可以通过像流水线一样写下来，最终变成一个几千行的函数、几万行的单个文件。这个时候甚至没有拆分成模块，也就更谈不上所谓依赖和解耦了。

忽然有一天，我们发现这种堆屎山的日子实在过的没有意思，开始研究怎么将一座大屎山拆成几个小屎山，然后再一点点清理干净。依赖的产生，就在我们一拆多的这个过程伴随出现的。

接口管理

当我们开始进行代码优化的时候，最先想到的就是将某些通用的功能抽象成单独的模块，通过提供接口这样的方式来给到需要的地方使用。

为了避免过度设计，我们会基于现有和可预见的需要进行设计。但日常的开发中，不可预见的问题定位和调整却占了大部分的时间。

例如，在做 To B 项目的时候，我们设计了一套完整的 API 给到对方，开始的时候大家都会按照这套接口来配合开发，其乐融融。突然有一天，老板拉了一个大客户，说这个客户的用户量会很大，必须要好好配合。老板一走，大客户马上化身甲方爸爸，说他们的接口已经写好了，友商都是按照他们的格式接入，都上线了。

遇到这种情况，通常会新增一个适配层，专门用于我们的服务和甲方爸爸之间的适配。

说了那么多，依赖在哪里呢？

依赖其实在接口设计完成的时候就出来了，虽然这是我们自己设计的接口，但它依赖于上游按照约定来调用。而上游有调整的时候，我们是需要跟随者适配或者调整的。

或者举个小一点的例子，我们在项目中使用了一个较出名的开源库。某一天该开源库升级版本了，新的版本不兼容旧的版本，同时声明旧的版本不会再继续维护了。这意味着如果我们不升级版本的话，后续旧的版本出现了 bug，我们只能自己啃源码来修复了。

这是来自于“甲方按照约定接口来调用服务”、“乙方按照约定接口来提供服务”的依赖。

状态管理

由接口管理产生的依赖通常来自外部，而应用内部也会有依赖的产生，常见的包括状态管理和事件管理。我们先来看看状态管理。

一个应用程序能按照预期正常运行，必然无法避免一些状态的管理。最简单的，生命周期就是一种状态。程序是否已经启动、功能是否正常运行、输入输出是否有变化，这些都会影响到程序的运行状态。

由于程序会有状态变化，因此我们的功能实现必然依赖程序的状态。例如，只有用户登录了才能进行更多的操作、订单产生了才可以进行撤销、界面渲染完成了用户才可以点击，等等。

从代码可读性和可维护性角度来看，面向对象编程近些年来还是稍胜于函数式编程，面向对象的设计本身就是状态设计的过程，而某个对象的运行结果，也会依赖于该对象的状态。

这是来自于对某个程序“按照预期运行”进行合理设计而产生的依赖。

功能管理

当我们根据功能将代码拆分成一个个模块之后，功能模块的管理也同样会产生一些依赖。

管理系统中最常见的就是面板的管理，对于每个面板来说，它应该只关心自身的状态。产品设计会要求我们在打开某个新的面板的时候，关闭其他面板；或是在点击面板以外的地方，关闭当前面板。这会涉及到面板与面板以外界面的通信，一般来说我们可以使用事件的方式来管理。每个面板在创建的时候，都需要监听外界的一个点击事件，并判断点击区域落在面板外面的时候，触发关闭。

某一天，产品提了个需求，所有的这些面板关闭的时候都要有一个动画效果，至于这个关闭动画的持续时间，要根据点击位置与面板的距离来计算。我们需要在点击事件触发的时候，把点击的位置告诉监听对象。

于是，我们全局搜了所有该类型事件的触发节点和监听节点，一一进行调整。

这是来自于对某个功能“不会发生变更”而产生的依赖。

依赖来自于约束

为了方便管理，我们设计了一些约定，并基于“大家都会遵守约定”的前提来提供更好、更便捷的服务。

举个例子，前端框架中为了更清晰地管理渲染层、数据层和逻辑处理，常用的设计包括 MVC、MVVM 等。而要使这样的架构设计发挥出效果，我们需要遵守其中的使用规范，不可以在数据层里直接修改界面等。

可以看到，依赖来自于对代码的设计。

依赖可以解耦吗

既然依赖来自于设计，为什么我们又常常说要进行模块间的解耦，降低模块间的依赖呢？

依赖的划分

我们先来看看一个问题，所有的依赖都需要解耦吗？

其实我们能看到，不合理的设计会导致代码间相互依赖，耦合严重。这种情况下，我们可以理解为产生了不合适的依赖。

而通常我们所说的设计解耦，则是通过合理的设计，恰到好处的职责和边界划分。此时，同样会产生一些约定，但这样的约定可以更好地管理我们的代码，此时可以理解为产生了合理的依赖。

因此，回到前面的疑问：既然依赖来自于设计，为什么我们要通过设计来降低依赖呢？显然，我们想要减少的，是不合理的依赖。而通过合理的设计，可以进行恰当的解耦。

无状态的函数式编程？

每个程序员对函数式编程都曾抱有过幻想，写多了面向对象编程的代码，对一些状态的管理和维护感到心烦。而无状态的函数式仿佛是白月光，可远观不可亵玩。

但即使是基于函数式编程设计的语言，写出来的功能也无法逃脱状态管理的命运。像 Clojure 编写的 Storm，也需要进行消息队列的管理、重启后服务的恢复等一系列状态管理。

在前端领域，React 同样基于函数式编程，但框架同样带有生命周期这样的状态。用 React 来实现的应用也依赖状态，因此同样产生了 Redux/Mobx 这样的工具来进行数据状态的管理工具。

应用程序无法离开状态的管理，是否意味着我们不需要函数式编程呢？并不是这样的。相反，我们需要对功能模块进行划分，划分出有状态和无状态的功能，来将状态管理放置到更小的范围，避免“牵一发而动全身”。

在这里，我们进行了状态有无的划分。

单向流的数据管理？

代码解耦的方式，其中也包括了使用单向数据流这种方式。

不管是 React 还是 Vue，都提供了单向数据流的管理工具。由于一个应用中，各个功能间都会有一些相互间的数据依赖，为了避免模块间的直接依赖，使用单向流的方式，可以将一些非模块内闭环的数据通过有序、单向的方式进行捆绑。通过这样的方式，模块之间的依赖解除了，调整为模块与数据流模块之间的依赖，代码的耦合程度得到缓解。

在这里，我们进行了模块内外数据的划分。

服务化

服务化，是系统解耦最常用的一种方式。

通过将功能进行业务领域的拆分，我们得到了不同领域的服务，常见的例如电商系统拆分成订单系统、购物车系统、商品系统、商家系统、支付系统等等。

而如今打得火热的“微服务”，也都是基于领域建模的一种实现方式。

在这里，我们进行了业务领域的划分。

模块化与依赖注入？

相比于针对系统设计的服务化，同样有针对功能设计的模块化。

在前端领域，同样可以根据功能拆分为表单功能、列表功能、面板功能等，通过给这些功能设置边界、封装成独立完整的模块，可以将功能与功能之间的依赖降到最低。同样的，根据功能划分的方式，我们还可以将功能拆分成渲染层、数据层、网络层这样的模块。

而配合依赖注入的方式，我们在使用这些功能的时候不再需要单独对这些功能的状态进行维护，同样实现了功能模块间的解耦。

7. 2025年，前端 如何与 AI 结合

TLDR

- **对于单元测试：** AI 考虑了非常全面的分支条件，但是运行就报错，因为一些基础依赖、mock 方法、API 的使用不够准确；
- **对于代码生成：** 第一次生成效果最好，在后续的讨论修改中逐渐变得上文不接下文，直到所有代码变得不可用；
- **对于代码补全：** Copilot 虽然很惊艳，但是在企业的业务开发过程中由于存在内部代码泄露风险，并不能随心所欲的使用；
- **对于代码助手：** 向它寻求帮助时，它编造不存在工具和 API 推荐给我；

总的来说，AI 代码生成这件事经过了一年时间的考验，仿佛变成了食之无味弃之可惜的鸡肋骨，程序员战战兢兢使用 AI 生成着代码，却惊喜地发现这东西一时半会还替代不了自己。

在日常开发中，AI 已被业界许多工具证明可以提高开发效率。但如果代码量快速膨胀，人类还改的过来吗？**如果随着 AI 生产力进一步提升，当代码检查成为效率的瓶颈时，大量未经人类把关的 AI 代码直接用在生产环境，人类真的敢用吗？**

AI 的表现如何？

考虑到 AI 已经被鼓吹可以替代人类，那么人类程序员能做到的代码重构效果，AI 也一定能做到，所以一开始在提示词中希望它在重构的过程中优化文件结构，移除冗余代码，同时保持原始逻辑不变。这是一个非常“普通且自信”的想法：既要、又要、还要。

于是沟通遇到的第一个问题出现了，**即使将复杂的任务拆分，人类也很难用自然语言明确告知模型，怎么样算优化文件结构，什么样的逻辑是冗余逻辑**，毕竟人类程序员还要经过反复的需求讨论才能逐渐确认需求的全部细节。于是一步到位的提示词方案被否定。

接下来对 AI 的要求是：文件对文件，逻辑对逻辑进行代码转换，不必做过程中的优化和调整。这样任务就简单了许多，降低了沟通成本，指令的下达变得明确而高效。

随着项目的推进，又遇到了类似的问题：**对于混杂在多任务中的简单小任务，AI 无法做到，无论是明确定义、增加示例、反复强调、给予鼓励**。比如特定第三方组件库的使用参数，JSX 条件渲染的实现等。这时问题可能出现在这些小任务合集触碰到了 AI 能力

的上限，除非针对特定任务精调，去提升它在任务中的表现。

去评判 AI 的表现，总的来说还是**沟通和能力边界**的问题。对于业务中使用的某个具体模型，它的能力边界不完全确定，由于模型内部逻辑原理的不可解释性，提示词工程师（Prompt Engineer，PE）们只能不断猜测、试验，而在这其中投入多少精力的度，是不确定的。另外考虑到 AI 的幻觉问题，如何评判“胜任”，如何确保 AI 每一次任务都“胜任”也是一个挑战。所以这充满不确定的沟通调试过程，影响了 AI 代码生成的效果，**较低的代码产物质量让它们暂时无法被信任地直接用在生产环境。**

总结

和人之间的信任一样，AI 的信任是逐渐建立的，从简单的任务到复杂的任务如果都能逐一胜任，那么人类对 AI 的信任就会增加。**对于 AI 出现的失误，并不是不被允许，只是这些失误需要是可解释的，并且有规律可规避的。**对 AI 更进一步的期待是，**其内部的运行逻辑是可解释的，其行为是可控的。**

我们期待这一天的到来，期待 AI 成为人类更加可信任的工作伙伴。

AI & 测试

你可能会觉得“啊，既然 AI 代码生成问题那么多，那么退一步让 AI 来测试吧”，但是，实际上 AI 测试当前也有着数不尽的问题。本小节将会详细拆解这些问题来逐一分析。

AI 与测试相关的问题

截止到目前，前端 + 大语言模型 + 测试的实践几乎为 0。如果你在 Github 搜索，仓库不超过 10 个，合计的 Star 数加起来 5 个都不到。

为什么？为什么感觉这么“前途无量”的场景连 Github 仓库都没几个？

这个问题很难用一句来解释清楚，但是我向你保证，看完本节，你一定会得到答案。

单元测试

先说实施起来最简单的单元测试。单元测试非常的直白，就是使用一段纯 JS 代码来测试另一段纯 JS 代码，比较结果，一样就成功不一样的就失败。这么简单的任务，强如 GPT-4，不会做不到吧？

能做到，但是不能完全做到。

其实在 AI + 单元测试，在 LLM 出现之前，业界就已经有很多很多的实践了，而在 LLM 后，又崛起了一批 LLM + 单元测试的试验品。但是我打赌读者一个也没听说过对不对？没听说过就对了，因为哪怕有了 LLM 的助力，效果依旧一言难尽，根本拿不出手。

接受率

在这里引入一个接受率的概念。简单来说就是 AI 生成了 100 个用例，哪些是可以直接使用的。假如有 80 个可以不经修改的直接使用，那么接受率就是 $80 / 100 = 80\%$ 。

但是这里有个非常尴尬事情——并不是说接受率 > 0% 那么就可以用，因为你依旧需要花精力在那些“不可用的用例”。

假如接受率是 40%，AI 生成了 100 个用例，你其实并不知道这 100 个用例中哪 40 个是正确的，而为了使用这 40 个正确的用例，你需要：

1. 想办法将可用的用例挑出来，并逐一验证
2. 将剩下不可用的用例一个一个改好
3. 重新整体检查，手动填补用例遗漏

假如接受率真的是 40%，这个过程将会是地狱，将远远超越你自己写 100 个用例的时间。

好了，那么不妨在此猜测一下，那么在 LLM 之前和之后，AI 单元测试的接受率是多少？

单元测试的进一步分析

通过接受率小节，我们了解的**接受率必须高到一定阈值才有意义**，否则就是单纯的把“写用例”变成了“调用例+改用例”。据我们的估计，接受率至少要达到 80% 以上，否则就完全是在帮倒忙。

- 而目前 LLM 单元测试 普遍接受率仅为 50% 不到，而如果是用 NLP 来代替 LLM，接受率将低到 30% 以下：

Pass@1 [%]						Pass@10 [%]					
	c++	go	java	Python	Javascript		c++	go	java	Python	Javascript
code-gen-6b-multi	11%	13%	15%	18%	13%	code-gen-6b-multi	25%	22%	31%	30%	26%
codegeex-13b	16%	12%	20%	22%	15%	codegeex-13b	31%	25%	36%	39%	31%
code-genius-6b	18%	17%	19%	33%	17%	code-genius-6b	32%	31%	40%	50%	38%
code-genius-16b	TBD	24%	TBD	35%	TBD	code-genius-16b	TBD	34%	TBD	55%	TBD
GPT3-Davinci	48%	32%	40%	48%	49%	GPT3-Davinci	TBD	TBD	TBD	TBD	TBD

@稀土掘金技术社区

这就好比有个实习生，写了 10 个页面，其中：

- 2 个页面看上去正常，实际上交互有各种问题和巧妙的 Bug
- 4 个页面运行正常，但是和上面这种页面混在一起了，你需要测试后才能找到他们
- 2 个页面打开就白屏，你简单修改几次它还是白屏，你也不知道具体为啥，需要慢慢调
- 2 个页面忘做了

这种实习生你要吗？因此，AI & 单元测试依旧要求程序员拥有极强的单测完善能力，它不能代替你来直接完成单元测试。

其他测试

单元测试包含了太多的细节和边界情况，那么我们把视野拉远，比如 E2E 测试，是不是会好一点呢？正好前端单测用的也不多。别说，你还真别说，当我们聚焦于 E2E 而不是单测后，各种问题确实是迎刃.....增加了~！

当你想使用 AI + E2E 后，第一件事就是——怎么让 AI 和浏览器交互。目前在这方面，业界没有任何的实践，**所有的胶水代码都必须由你来开发。**

目前 GPT 确实有检索和理解网页的能力，但是仅限于内容的理解。无法做到对于前端页面交互的理解，比如在选择框来选择某项

在单元测试中，我们可以很容易的“报错后直接发给 AI，让 AI 多试几次”，但是这个交互流程在 E2E 情景下是没有相关实践的，**其中伴随很多的开发工作。**

同样的，AI 对于一个网页的理解是非常有限的，他并不知道一个用户交互是错误的还是正确的，你可能还需要传递给 AI 一些背景资料，比如产品文档、测试用例、交互图、代码。这部分的序列化工作也是一个全新的领域，**所有的胶水代码都必须由你来开发。**

如果你不是在一个大型 IT 公司，开发胶水代码成本是难以接受的。它可能要花费一年的时间，并且过程中没有任何阶段性产出，且部分功能随时可能被新一代的 LLM 所代替。

总结

通过单元测试 和 E2E 测试的分析，相信读者已经明白了的当前 AI & 测试面临的主要挑战：

- 如何序列化让 AI 理解现状上下文
- 过低接受率导致的额外人工介入的成本

但是我们相信这些这些问题都是有解的。可能是更加聪明的 LLM 提高了接受率，可能是规范的合作方合作流程，比如测试用例写的足够可读，并能直接导出为表格发送给 LLM，也可能是某个公司大笔一挥，决定花一年来开发一个 AI E2E 测试框架。

现在，软件测试的世界在等待一位英雄，他手持 LLM 的利剑，彻底斩除软件开发的质量痛点。而那位英雄，会不会是正在阅读本文的你呢？

AI & 辅助CR

除了开发与测试，换个角度看问题，**如果让 AI 来当我们的 CR 助手呢？**

代码审查（CodeReview）是软件开发过程中的重要环节，Code Review 是指开发人员对彼此的代码进行审查和评估的过程。通过 Code Review，团队成员可以相互检查代码的正确性、可读性、可维护性以及是否符合编码规范等方面的问题。可以帮助开发者发现并修复代码中的错误和缺陷，提高代码质量，减少技术债务。

传统的代码审查由人工完成，团队在 Code Review 上花费了大量时间，且容易出现疏漏。随着大型语言模型（LLM）的发展，LLM 在代码审查领域得到了越来越多的关注。LLM 可以通过对代码的语义理解，快速识别代码中的错误和缺陷，从而提高代码审查的效率和准确性，以下是 AI Code Review 的几个好处：

- **自动化和效率：**与传统的人工 Code Review 相比，AI Code Review 具有**更高的自动化程度**，可以大大提高审查的效率。
- **捕捉更多问题：**AI Code Review 工具可以检测出更多的代码问题，**包括潜在的性能问题、代码重复、安全漏洞等**，帮助开发人员更全面地改进代码。
- **减少人为偏差：**由于人为因素，传统的 Code Review 可能存在一些主观偏差。而 AI Code Review 可以基于大量的事实和规则进行评估，减少主观因素的影响，**提供更客观的评价结果**。
- **持续集成与部署：**AI Code Review 工具可以与**CI/CD 相结合**，实现自动的代码审查和反馈，帮助开发团队更好地管理代码质量。

AI 在 Code Review 的场景

LLM 在代码审查领域的应用主要有以下几个方面：

- **补充 CR 信息：**根据 CR 的 Diff 内容，生成 CR 的表述、标题、CR 总结，方便人工短时间了解代码变更关键信息。
- **基本代码问题：**变量名、函数名的语义化、可读性、可维护性、可扩展性等。通过分析代码的结构、语法和语义特征，工具可以自动判断代码的质量等级，为开发团队提供改进建议。
- **代码性能问题：**AI 代码评审工具可以分析代码的执行效率，发现潜在的性能瓶颈和优化点。例如，工具可以识别循环嵌套、冗余计算等问题，并提供优化建议，提高代码的执行效率。
- **逻辑问题：**比如边界问题、条件判断、循环控制的合理性。
- **代码安全问题：**AI 代码评审工具可以用于识别潜在的安全漏洞和风险。例如，工具可以分析代码中的敏感数据处理、输入验证、权限控制、SQL 注入，XSS 等方面，发现可能存在的安全问题，并提供修复建议。

当前应用

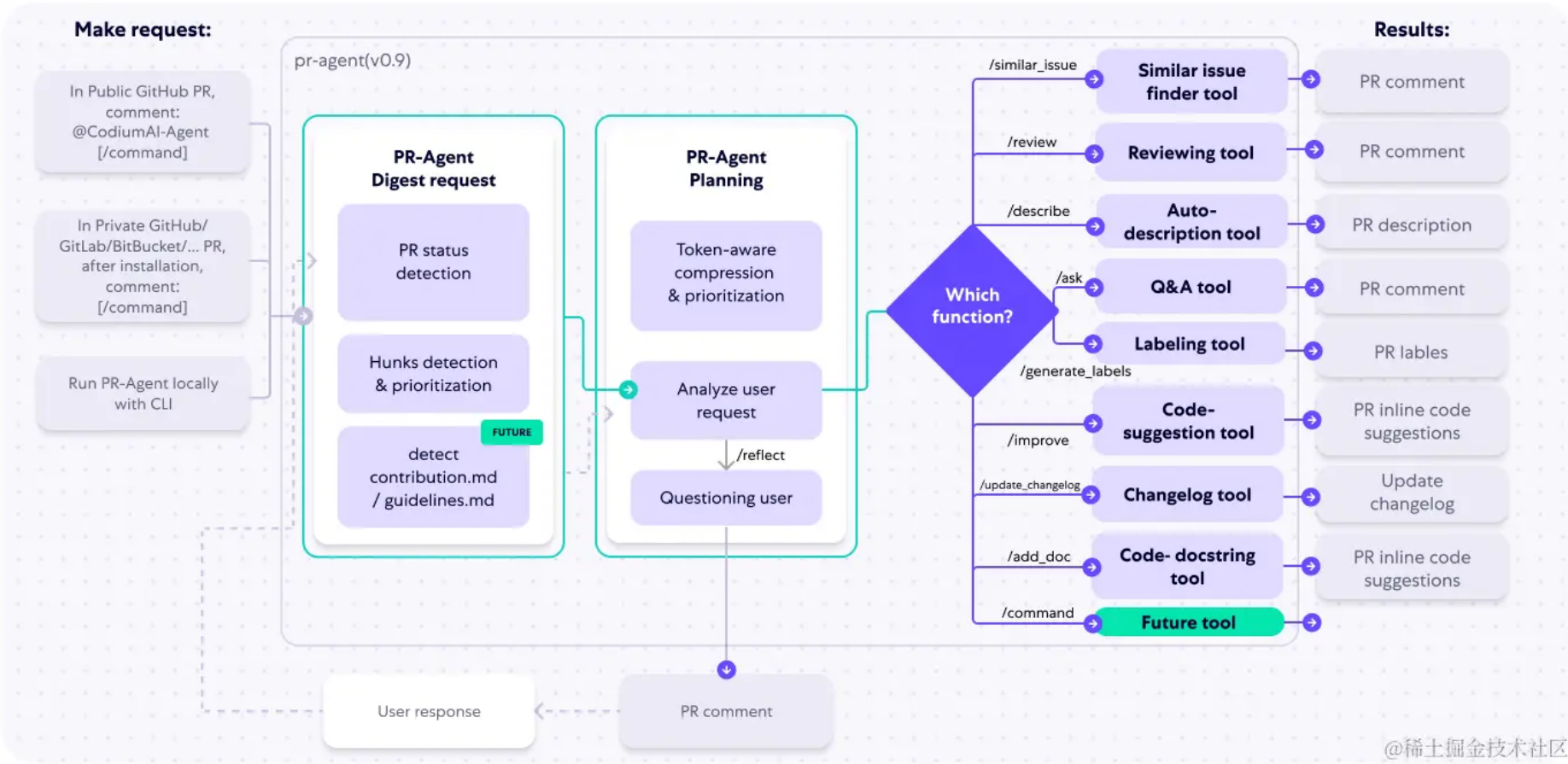
当前业界有不少 AI Code Review 的能力应用，除了大家熟知的 Github Copilot，开发者对其进行提问可以帮助完成 Code Review 的动作，我们再看一下还有哪些成熟的应用。

ChatGPT-CodeReview

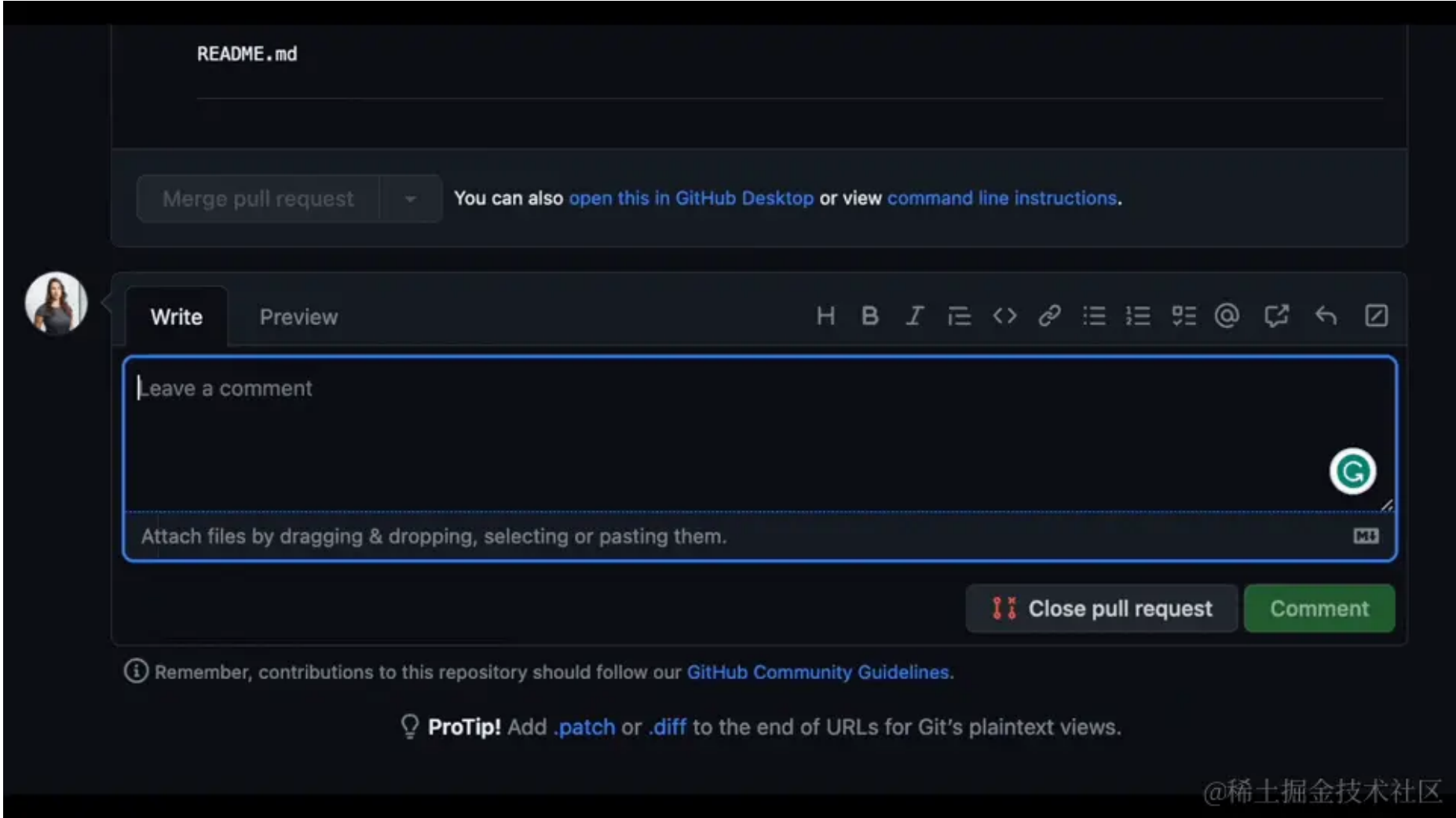
通过调用 Chagpt 的 Open API 以 Github Action 的形式集成到 Github 里面，机器人会自动进行代码审查，审查信息将显示在 pr timeline / file changes 部分。在 `git push` 更新 PR 之后，CR bot 将重新审查更改的文件：

CodiumAI PR-Agent

CodiumAI PR-Agent：一款基于 Chatgpt 的开源工具，可帮助高效审查和处理拉取请求。它自动分析 pull request，并提供多种类型的命令：自动生成 PR 描述、可调整的反馈、问题解答、代码建议、更新 CHANGELOG.md 文件、查找相似问题、自动添加文档、生成自定义标签和自动分析 PR 变更。



@稀土掘金技术社区



@稀土掘金技术社区

总结

如果你决定开始使用 AI 能力作为团队 CR 助手，**请注意：**

- 由于 LLM 本身的 Token 上下文限制，**当 MR 超出最大的 Context 上下文限制时，会造成信息丢失导致 Review 结果不全面。**
- 由于 LLM 回答的随机性，同一份代码多次 Review 可能有不同的结果，以及 Review 的结论会偏啰嗦，不容易抓出重点问题，**所以在提示词里面可以多强调关注重点问题**，比如 Prompt 加入 “作为一名代码 review 专家、对 Gitlab 的 MR 进行代码 review。只关注代码性能、代码安全问题、严重的逻辑错误”
- 如果要针对团队业务场景或基于团队研发规范进行 AI Review，可以将团队的业务背景信息规范等作为知识库对模型进行微调，对于担心代码泄漏的场景，还可以对开源的 LLM 进行私有化部署作为企业内的方案。

一言以蔽之，当前只是辅助人工 Review 帮助快速发现一些问题，减轻人工的简单重复的劳动，但无法完全取代人，**最终还得由人作判断**，但我们可以用节省下来的人力关注到更高优的事情上面。

AI & 低代码

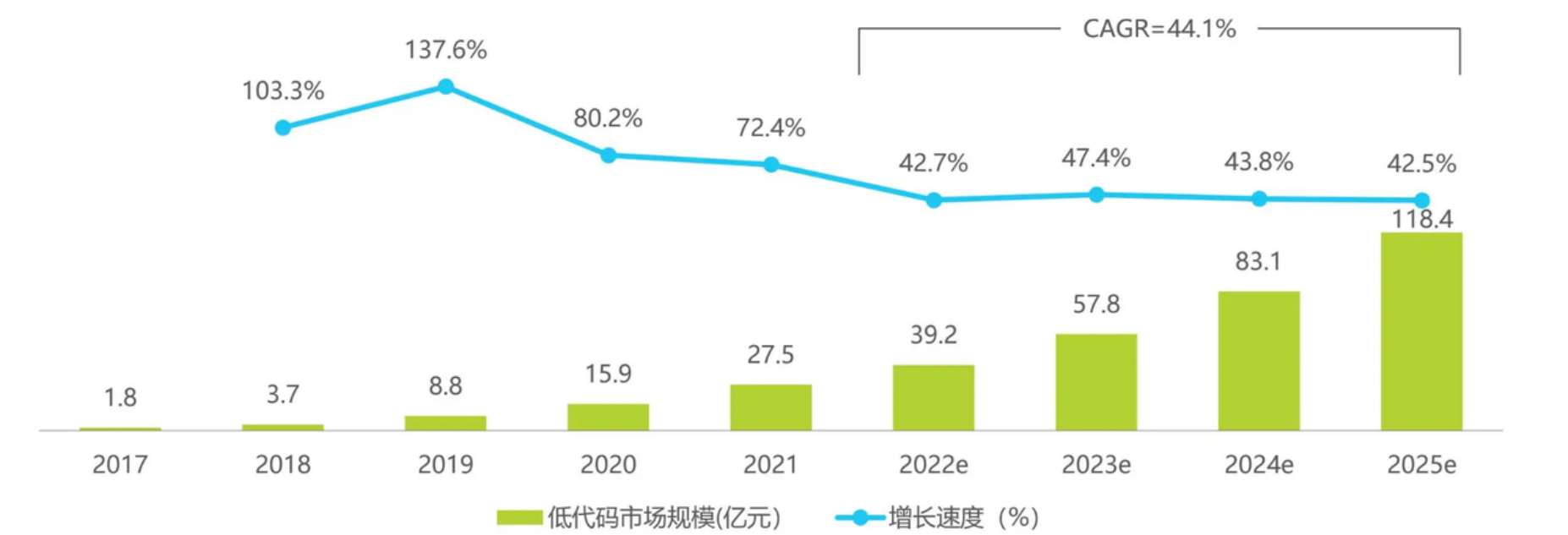
前两年火爆的低代码概念，似乎和 AI 有不错的“缘分”，两者结合往往能达到一加一大于二的效果。

什么是低代码

低代码开发平台（英语：Low-Code Development Platform，简称 LCDP），是一种方便产生应用程序的平台软件，软件会让用户以图形化接口以及配置编写程序，而不是用传统的程序设计作法。此平台是针对某些种类的应用而设计开发的，例如数据库、业务过程、以及用户界面（例如网页应用程序）。这类平台一般可以产生完整且可运作的应用 代码，但在一些特殊的情形下仍需要编写程序。

低代码概念最早出现在 1982 年《无程序员的应用程序开发》一书中，美国低代码产品的研究和实践过程较长，积累了较丰富的经验。中国最早出现低代码平台大概是在 2014 年，产品的形态从最初的数据库交付，到数据集结构搭建逐渐演变抽象出各种流程引擎、可视化界面等产品能力，应用能力也从 BPM 延伸到 ERP、CRM 等大型复杂应用，整个行业经历了 2017-2020 年的快速发展阶段，市场增速开始放缓，但相对于其他企业服务赛道仍处于高增长阶段，预计 2025 年中国低代码行业市场规模将达到 118.4 亿。

2017-2025年中国低代码行业市场规模及增速

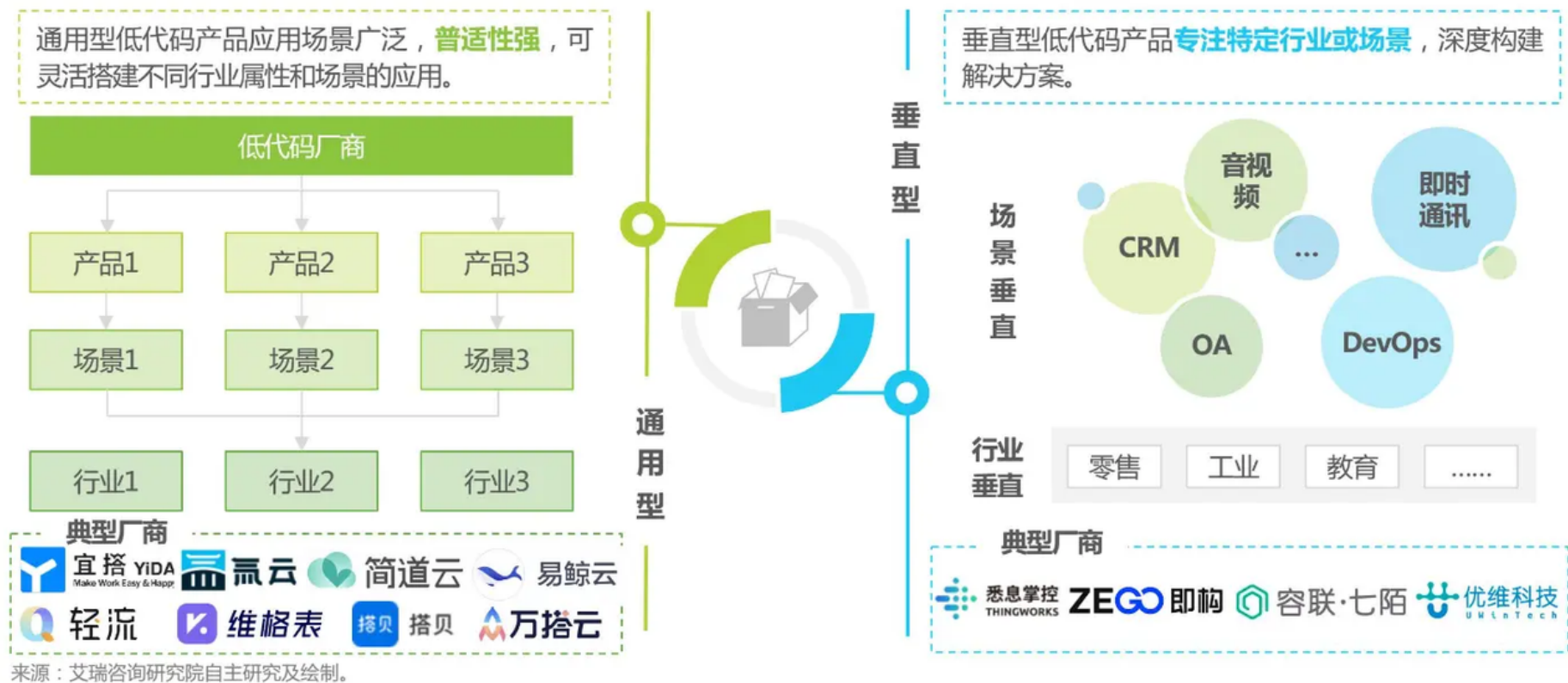


注释：本报告市场规模统计仅包含通过提供独立低代码平台并产生营收的厂商，低代码平台仅供内部产品使用的部分未包含在本报告研究范围。
来源：根据专家访谈、公司财报以及艾瑞市场模型推算获得，艾瑞咨询研究院自主研究及绘制。
©2022.8 iResearch Inc. www.iresearch.com.cn @稀土掘金技术社区

存在问题

但低代码自出现以来，一直还是饱受质疑，核心的点在于通过低代码的可视化搭建大大降低了建设成本，且使用群体从开发人员逐步向业务人员渗透，但低代码本身是存在一定的上手门槛，包括各类物料的用法、如何实现与后端接口通信、物料之间如何配置交互等等，这使得业务人员无法快速上手完成页面搭建，而开发人员需要学习特定搭建规则和限制，可能不如实际开发来的快。
在不同类型的低代码产品的相关的限制又会有所差异，从应用场景上可以划分通用型和垂直型

通用型和垂直型产品特征对比



©2022.3 iResearch Inc. www.iresearch.com.cn @稀土掘金技术社区

- 对于垂直型低代码产品，深耕特定行业或特定的场景，提供当前垂类更具有专业性的解决方案和搭建能力，在具体的搭建场景中专业性贴合更深，会存在复杂搭建场景，如公式、模型、数据等，**门槛更高，搭建依赖的前置输入也要更多。**
- 而对于通用型，由于覆盖各行业各场景，沉淀了大量的行业模版和解决方案，**虽然搭建门槛相对垂直型要更低，但内容覆盖面更广，大量的物料和模版不仅造成了用户选择压力，也提升了搭建的学习负担。**

低代码产品本身的易用性有待进一步提升，包括底层技术框架和可视化界面的呈现，另外低代码相关的培训机制有待完善，培训不足导致用户对于低代码产品的接纳度低，用户本身对于繁重的产品使用学习的意愿不足，也会大大阻碍低代码产品的推广和应用。

总结

当然 AIGC 与低代码的融合还处于早期阶段，当前应用范围也主要还是在一些容错率较高的场景，通过大语言模型可能不能完全理解用户的真实需求，这需要庞大的上下文做支撑，虽然融合方向仍然有诸多问题亟需解决，但未来低代码平台必将会全面拥抱 AIGC，这也意味着我们需要重新去思考低代码平台的交互界面的设计，充分利用自然语言交互特点，提升用户体验。

另外数据安全风险也是在商业化产品中必须要考量的一点，因此多数企业都在尝试走自研大模型的路子，然而这并不是一件容易的事。它需要大量高质量数据的收集、清洗和标注，以及昂贵的计算资源来支持模型训练。这也意味着未来 AI 加持下的低代码平台的竞争成本会更高，会更加聚焦在有雄厚实力的头部企业。

除了 AIGC +低代码融合之外，还有一方观点认为既然 AIGC 能力越来越强，是否会完全替代低代码，直接生成原始应用？我们认为至少在可以预见很长范围内低代码不会被完全替换掉，就像上面几部分提到的，大模型通过语义直接生成代码并不能保证其完全的准确性，且代码可用性需要大量的人工校正。而低代码融合 AIGC 可以通过语义生成模型，加速需求分析工作进程，可以让 AI 更快的进入业务。

8. 师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

8.1 项目实战内容

github: <https://github.com/encode-studio-fe>

[前端实战课分享汇总](#)（可咨询班主任要密码权限）

8.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



8.3 专业一对一

近期一对一

- 📅 (2024-05-31) xxx-2.5年-React-24K
- 📅 (2024-05-23) xxx-1年-React-15k
- 📅 (2024-05-19) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历（For 付费学员）

8.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

8.5 近期报名学员案例

